

lerobot 机械臂 windows 配置测试

0. 安装包

lerobot.zip

Miniconda3-py310_25.7.0-2-Windows-x86_64.zip

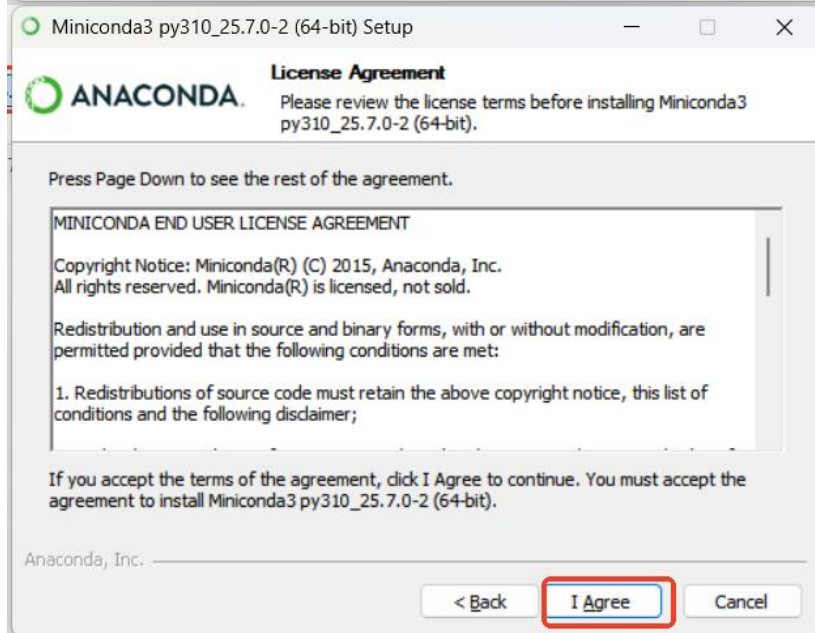
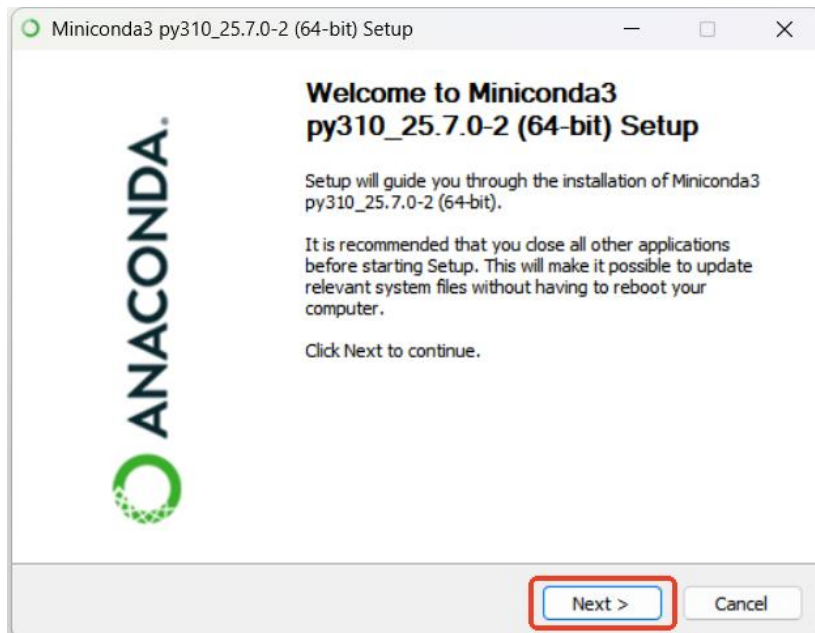
MobaXterm_Installer_v22.1.zip

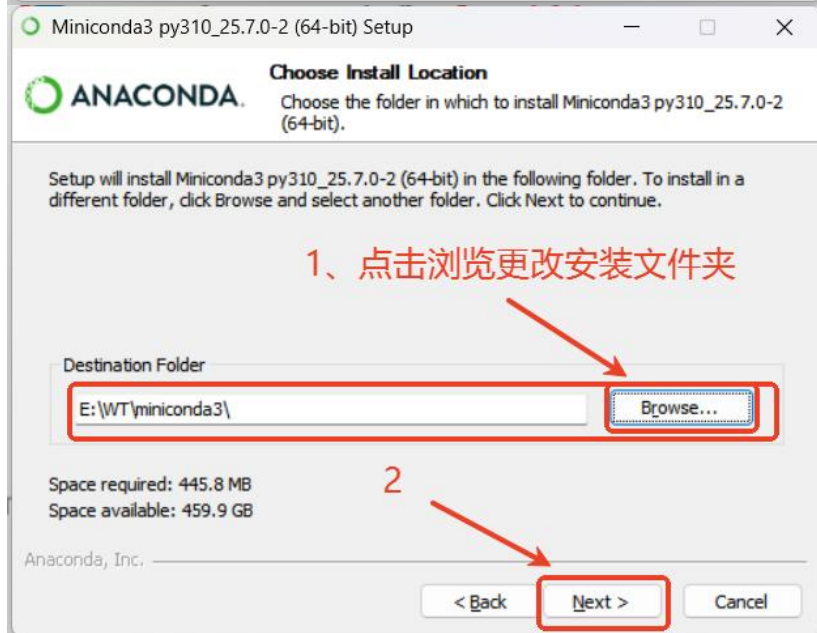
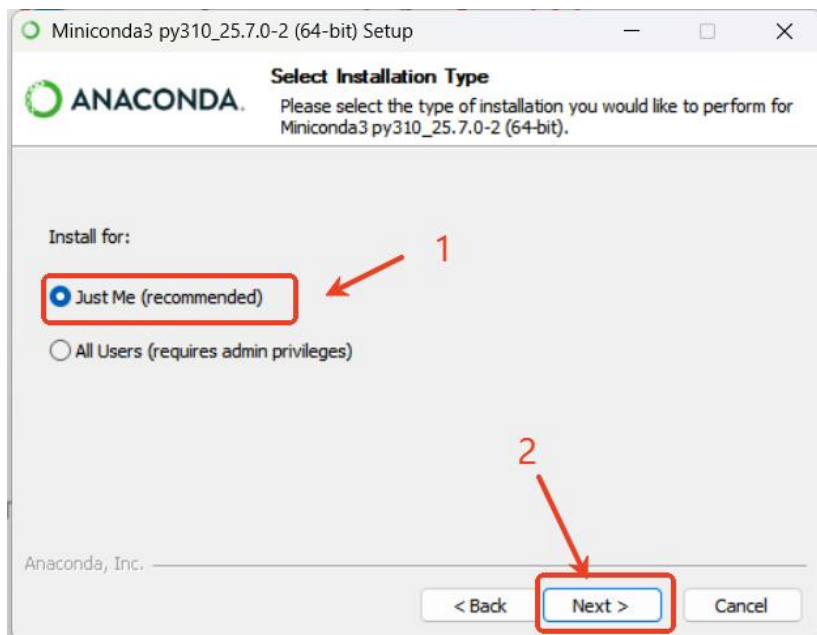
1.miniconda 安装配置

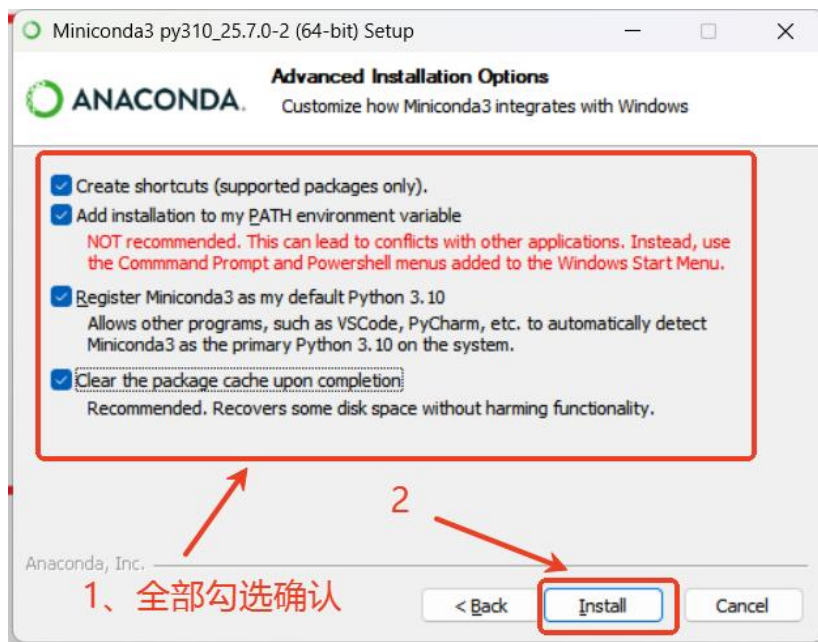
1.1miniconda 安装

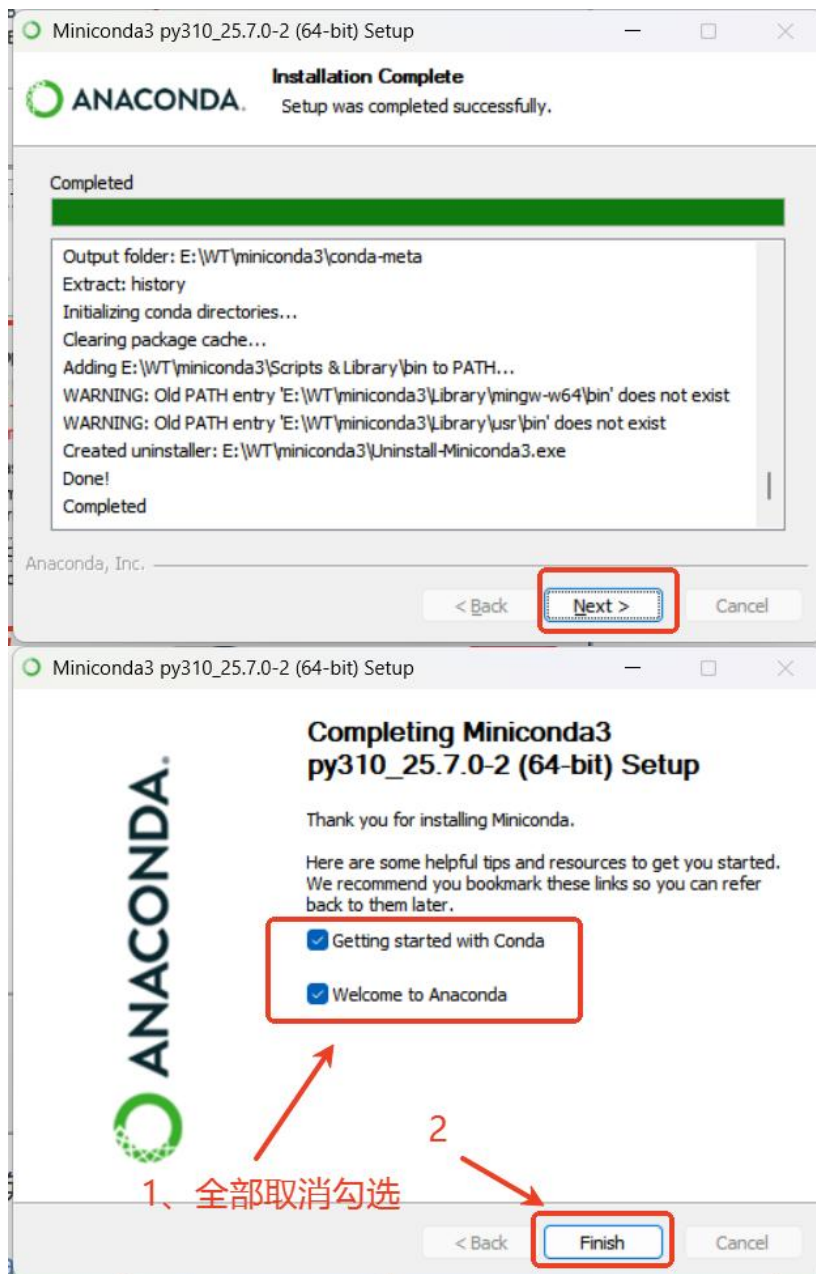
双击打开

名称	修改日期	类型	大小
 Miniconda3-py310_25.7.0-2-Window...	2025/12/16 9:06	应用程序	89,258 KB



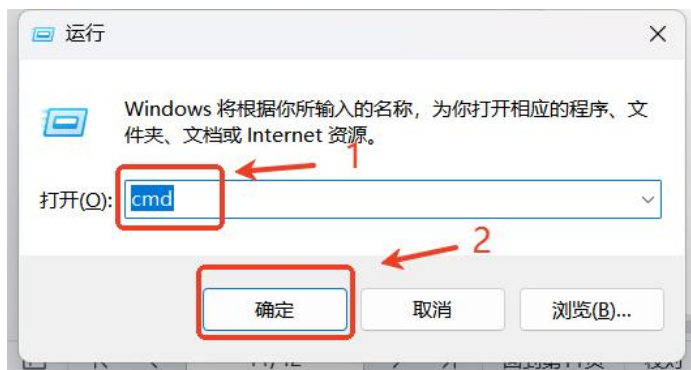






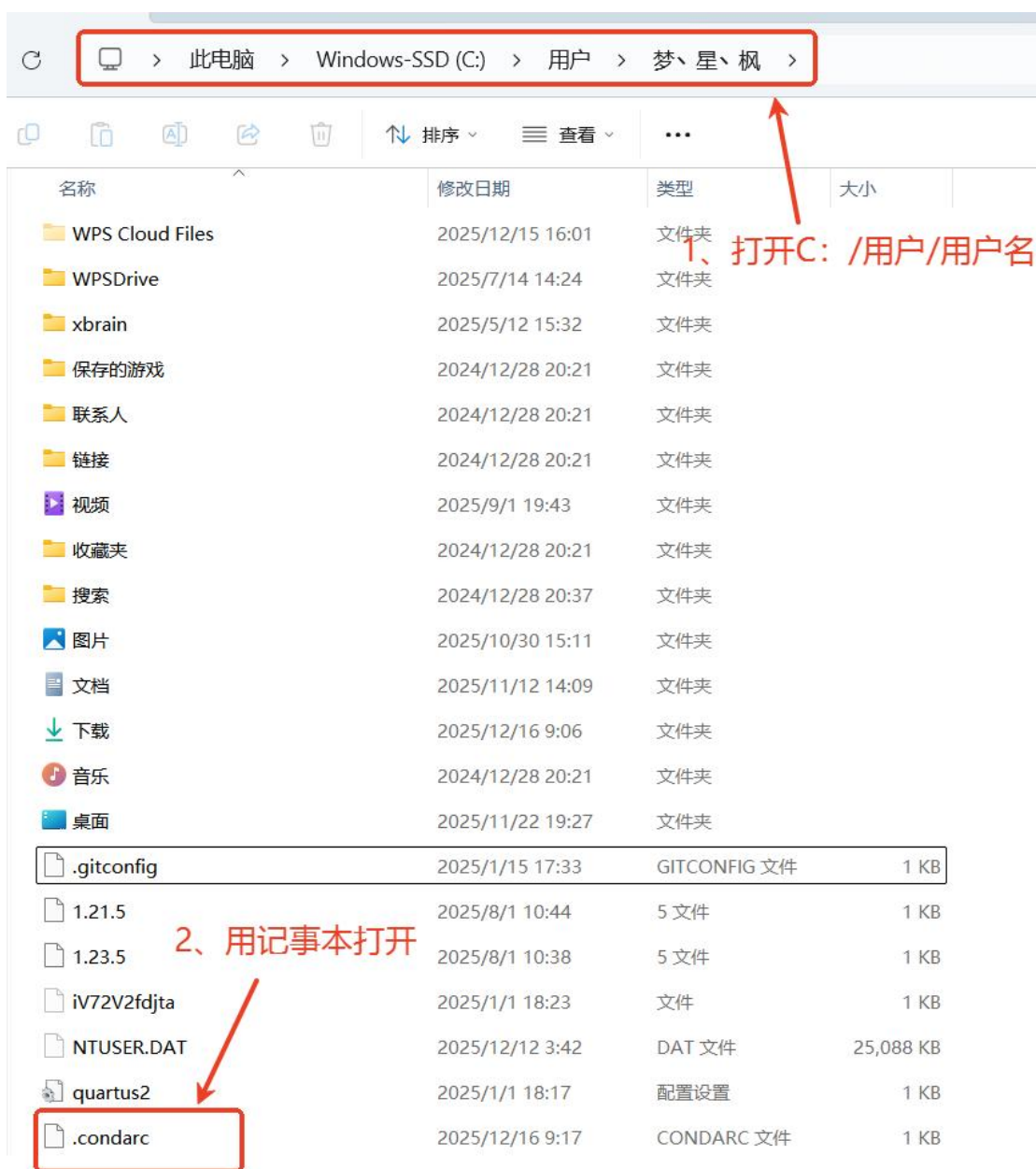
1.2minicoda 换源

按 win+R 打开运行提示框，输入 cmd 确认

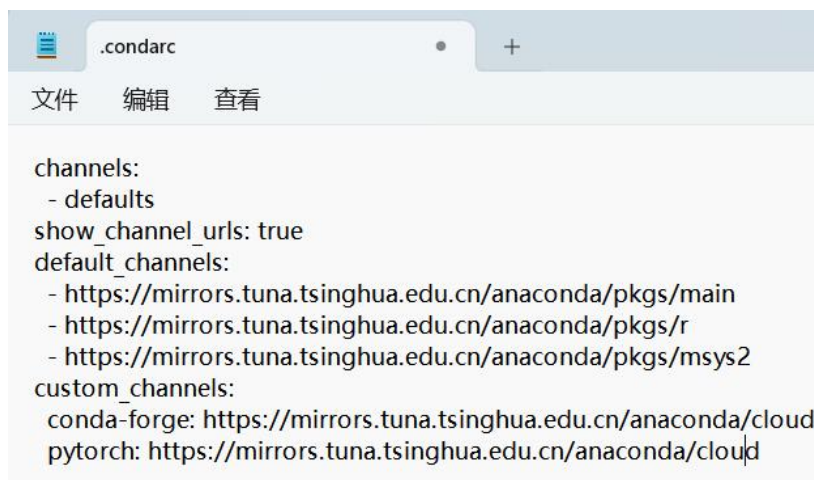


在打开的命令行窗口中输入 `conda config --set show_channel_urls yes` 生成 `.condarc` 文件

```
C:\Users\梦、星、枫>conda config --set show_channel_urls yes
```



```
channels:
  - defaults
show_channel_urls: true
default_channels:
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/r
  - https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/msys2
custom_channels:
  conda-forge: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
  pytorch: https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud
```



```
conda clean -i
```

```
C:\Users\梦、星、枫>conda clean -i
Will remove 1 index cache(s).
Proceed ([y]/n)? y
```

输入 `activate` 检查安装是否成功，路径前面出现 `(base)` 则为成功

```
C:\Users\梦、星、枫>activate
(base) C:\Users\梦、星、枫>d
```

输入 `conda deactivate` 即可退出 base 环境


```
(base) C:\Users\梦、星、枫>conda deactivate  
C:\Users\梦、星、枫>|
```

2. 虚拟环境配置

```
conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge
```

```
C:\Users\梦、星、枫>conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-forge  
Channels:  
- conda-forge  
- defaults  
Platform: win-64  
Collecting package metadata (repodata.json): done  
Solving environment: done  
  
## Package Plan ##  
  
environment location: E:\WT\miniconda3\envs\lerobot  
  
added / updated specs:  
- ffmpeg=7.1.1  
- python=3.10.18
```

输入 `y` 按 `enter` 确认

```
wheel      conda-forge/noarch::wheel-0.45.1-pyhd8ed1ab_1  
x264       conda-forge/win-64::x264-1!164.3095-h8ffe710_2  
x265       conda-forge/win-64::x265-3.5-h2d74725_3  
zstd       conda-forge/win-64::zstd-1.5.7-h534d264_6  
  
Proceed ([y]/n)? y
```

出现如下界面则为创建完毕

```
done  
#  
# To activate this environment, use  
#  
#     $ conda activate lerobot  
#  
# To deactivate an active environment, use  
#  
#     $ conda deactivate
```


输入 `conda env list` 检查环境是否存在

```
C:\Users\梦、星、枫>conda env list

# conda environments:
#
base                  E:\WT\miniconda3
lerobot               E:\WT\miniconda3\envs\lerobot
```

输入 `conda activate lerobot` 进入环境

```
C:\Users\梦、星、枫>conda activate lerobot
(lerobot) C:\Users\梦、星、枫>|
```

输入 `cd /d E:` 切换到 E 盘，再输入 `cd E:\WT\lerobot` 进入指定文件夹(红色部分为 `lerobot` 压缩包解压后的路径和盘符)

```
(lerobot) C:\Users\梦、星、枫>cd /d E:
(lerobot) E:\>cd E:\WT\lerobot
(lerobot) E:\WT\lerobot>|
```

```
pip install -e ".[feetech]" -i https://pypi.tuna.tsinghua.edu.cn/simple
```

```
(lerobot) E:\WT\lerobot>pip install -e ".[feetech]" -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Obtaining file:///E:/WT/lerobot
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Collecting datasets<4.2.0,>=4.0.0 (from lerobot==0.4.3)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/f4/c8/09012ac195a0aab58755800d2efdc0e7d5905053509f12cb5d136c911cda/datasets-4.1.1-py3-none-any.whl (503 kB)
Collecting diffusers<0.36.0,>=0.27.2 (from lerobot==0.4.3)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/2a/2e/38d9824f8c6bb048c5ba21c6d4da54c29c162a46b58b3ef907a360a76d3e/diffusers-0.35.2-py3-none-any.whl (4.1 MB)
  4.1/4.1 MB 7.7 MB/s 0:00:00
Collecting huggingface-hub<0.36.0,>=0.34.2 (from huggingface-hub[cli,hf-transfer]<0.36.0,>=0.34.2->lerobot==0.4.3)
```

3. 主从臂机械组装

3.1 主从臂舵机分配

	标准版	专业版
ST-3215-C001	7	1

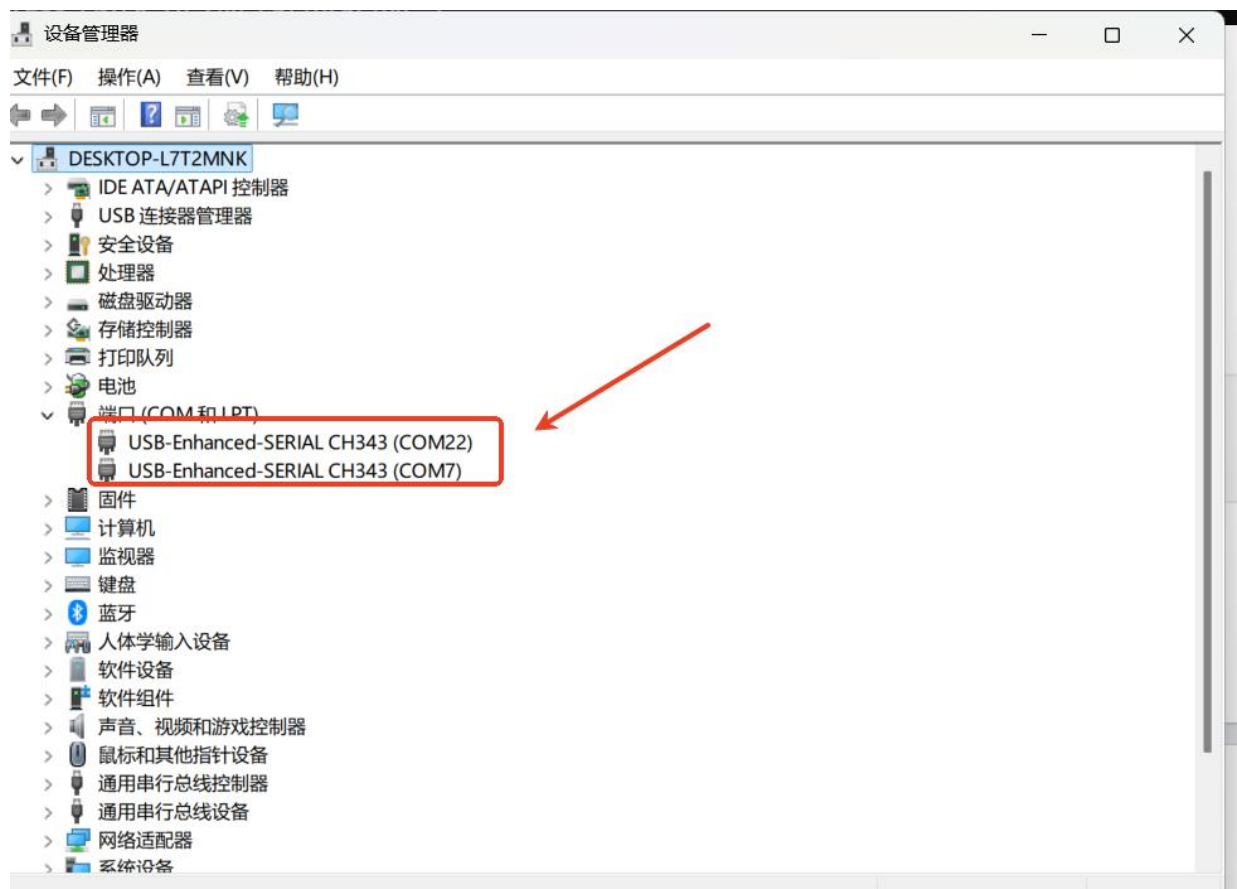
ST-3215-C044	2	2
ST-3215-C046	3	3
ST-3215-C018/C047	0	6

3.2 主从臂安装视频

详细请观看压缩包内组装视频(路径: 1. 课程资料/视频教程/)

4. 舵机校准

打开设备管理器即可观看端口号(不知道哪个端口号对应哪个机械臂请拔出, 观看设备管理器记录端口号后再插上)



```
cd WT\lerobot

lerobot-setup-motors --robot.type=sol101_follower --robot.port=COM7


cd WT\lerobot

lerobot-setup-motors --teleop.type=sol101_leader --teleop.port=COM22
```

```
Connect the controller board to the 'gripper' motor only and press enter.
'gripper' motor id set to 6
Connect the controller board to the 'wrist_roll' motor only and press enter.
'wrist_roll' motor id set to 5
Connect the controller board to the 'wrist_flex' motor only and press enter.
'wrist_flex' motor id set to 4
Connect the controller board to the 'elbow_flex' motor only and press enter.
'elbow_flex' motor id set to 3
Connect the controller board to the 'shoulder_lift' motor only and press enter.
'shoulder_lift' motor id set to 2
Connect the controller board to the 'shoulder_pan' motor only and press enter.
'shoulder_pan' motor id set to 1
```

5. 主从臂校准

```
del /f C:\Users\梦、星、枫\.cache\huggingface\lerobot\calibration\robots
del /f C:\Users\梦、星、枫\.cache\huggingface\lerobot\calibration\teleoperators
```

如果你维修或者更换过机械臂零件，请完全删除 C:/用户名
/.cache/huggingface/lerobot/calibration/robots 或者 C:/用户名
/.cache/huggingface/lerobot/calibration/teleoperators 下的文件并重新校准机械臂，否者会出现
报错提示，校准的机械臂信息会存储该目录下的 json 文件中。

```
lerobot-calibrate --robot.type=sol101_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm
```

```
(lerobot) E:\WT\lerobot_xidi>lerobot-calibrate --robot.type=so101_follower --robot.port=COM7 --robot.id=my_awesome_follower_arm
INFO 2025-12-16 11:39:20 calibrate.py:74 {'robot': {'calibration_dir': None,
  'cameras': {},
  'disable_torque_on_disconnect': True,
  'id': 'my_awesome_follower_arm',
  'max_relative_target': None,
  'port': 'COM7',
  'use_degrees': False},
  'teleop': None}
INFO 2025-12-16 11:39:20 follower.py:104 my_awesome_follower_arm S0101Follower connected.
INFO 2025-12-16 11:39:20 follower.py:121
Running calibration of my_awesome_follower_arm S0101Follower
Move my_awesome_follower_arm S0101Follower to the middle of its range of motion and press ENTER....
Move all joints sequentially through their entire ranges of motion.
Recording positions. Press ENTER to stop...

-----
NAME      | MIN | POS | MAX
shoulder_pan | 856 | 2003 | 3379
shoulder_lift | 721 | 796 | 3316
elbow_flex  | 871 | 3056 | 3056
wrist_flex  | 940 | 2928 | 3238
wrist_roll  | 131 | 2051 | 3946
gripper     | 1405 | 1410 | 2904
Calibration saved to C:\Users\梦、星、枫\.cache\huggingface\lerobot\calibration\robots\so101_follower\my_awesome_follower_arm.json
INFO 2025-12-16 11:41:17 follower.py:230 my_awesome_follower_arm S0101Follower disconnected.
```

```
lerobot-calibrate --teleop.type=so101_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm
```

```
(lerobot) E:\WT\lerobot_xidi>lerobot-calibrate --teleop.type=so101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm
INFO 2025-12-16 11:47:01 calibrate.py:74 {'robot': None,
  'teleop': {'calibration_dir': None,
  'id': 'my_awesome_leader_arm',
  'port': 'COM22',
  'use_degrees': False}}
INFO 2025-12-16 11:47:01 01_leader.py:82 my_awesome_leader_arm S0101Leader connected.
INFO 2025-12-16 11:47:01 01_leader.py:99
Running calibration of my_awesome_leader_arm S0101Leader
Move my_awesome_leader_arm S0101Leader to the middle of its range of motion and press ENTER....
Move all joints sequentially through their entire ranges of motion.
Recording positions. Press ENTER to stop...

-----
NAME      | MIN | POS | MAX
shoulder_pan | 766 | 2045 | 3294
shoulder_lift | 813 | 824 | 3205
elbow_flex  | 894 | 3013 | 3093
wrist_flex  | 833 | 2882 | 3181
wrist_roll  | 89 | 2038 | 3921
gripper     | 1559 | 1572 | 2838
Calibration saved to C:\Users\梦、星、枫\.cache\huggingface\lerobot\calibration\teleoperators\so101_leader\my_awesome_leader_arm.json
INFO 2025-12-16 11:48:20 1_leader.py:156 my_awesome_leader_arm S0101Leader disconnected.
```

6. 主从臂跟随

6.1 没有摄像头

```
lerobot-teleoperate --robot.type=so101_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm --teleop.type=so101_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm --robot.disable_torque_on_disconnect=true
```

```
(lerobot) E:\WT\lerobot_xidi>lerobot-teleoperate --teleop.type=s0101_leader --teleop.port=COM22 --teleop.id=my_awesome_leader_arm --robot.type=s0101_follower --robot.port=COM7 --robot.id=my_awesome_follower_arm
INFO 2025-12-16 13:44:10 eoperate.py:188 {'display_data': False,
'fps': 60,
'robot': {'calibration_dir': None,
'cameras': {},
'disable_torque_on_disconnect': True,
'id': 'my_awesome_follower_arm',
'max_relative_target': None,
'port': 'COM7',
'use_degrees': False},
'teleop': {'calibration_dir': None,
'id': 'my_awesome_leader_arm',
'port': 'COM22',
'use_degrees': False},
'teleop_time_s': None}
INFO 2025-12-16 13:44:10 01_leader.py:76 Mismatch between calibration values in the motor and the calibration file or no calibration file found
Press ENTER to use provided calibration file associated with the id my_awesome_leader_arm, or type 'c' and press ENTER to run calibration:
INFO 2025-12-16 13:44:11 01_leader.py:95 Writing calibration file associated with the id my_awesome_leader_arm to the motors
INFO 2025-12-16 13:44:12 01_leader.py:82 my_awesome_leader_arm S0101Leader connected.
INFO 2025-12-16 13:44:12 follower.py:104 my_awesome_follower_arm S0101Follower connected.
Teleop loop time: 16.68ms (60 Hz)
INFO 2025-12-16 13:45:13 1_leader.py:156 my_awesome_leader_arm S0101Leader disconnected.
INFO 2025-12-16 13:45:13 follower.py:230 my_awesome_follower_arm S0101Follower disconnected.
```

6.2 有摄像头

使用 orbbec 深度相机

cd /WT

git clone <https://github.com/orbbec/pyorbbecsdk.git>

```
lerobot-find-cameras opencv
```

```
lerobot-teleoperate --teleop.type=s0101_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm --robot.type=s0101_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv, index_or_path: 0,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'},front: {type: opencv, index_or_path: 1,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --display_data=true --
robot.disable_torque_on_disconnect=true
```

```

    raise ConnectionError(
ConnectionError: Failed to open OpenCVCamera(0).Run 'lerobot-find-cameras opencv' to find available cameras.
[2025-12-16T06:58:29Z INFO egui_wgpu] There were 2 available wgpu adapters: {backend: Vulkan, device_type: DiscreteGpu,
name: "NVIDIA GeForce RTX 3050 Laptop GPU", driver: "NVIDIA", driver_info: "546.30", vendor: NVIDIA (0x10DE), device: 0
x25E2}, {backend: GL, device_type: Other, name: "NVIDIA GeForce RTX 3050 Laptop GPU/PCIe/SSE2", driver_info: "4.6.0 NVID
IA 546.30", vendor: NVIDIA (0x10DE)}

Welcome to Rerun!

This open source library collects anonymous usage data to
help the Rerun team improve the library.

Summary:
- We only collect high level events about the features used within the Rerun Viewer.
- The actual data you log to Rerun, such as point clouds, images, or text logs,
  will never be collected.
- We don't log IP addresses.
- We don't log your user name, file paths, or any personal identifiable data.
- Usage data we do collect will be sent to and stored on servers within the EU.

For more details and instructions on how to opt out, run the command:

rerun analytics details

As this is this your first session, _no_ usage data has been sent yet,
giving you an opportunity to opt-out first if you wish.

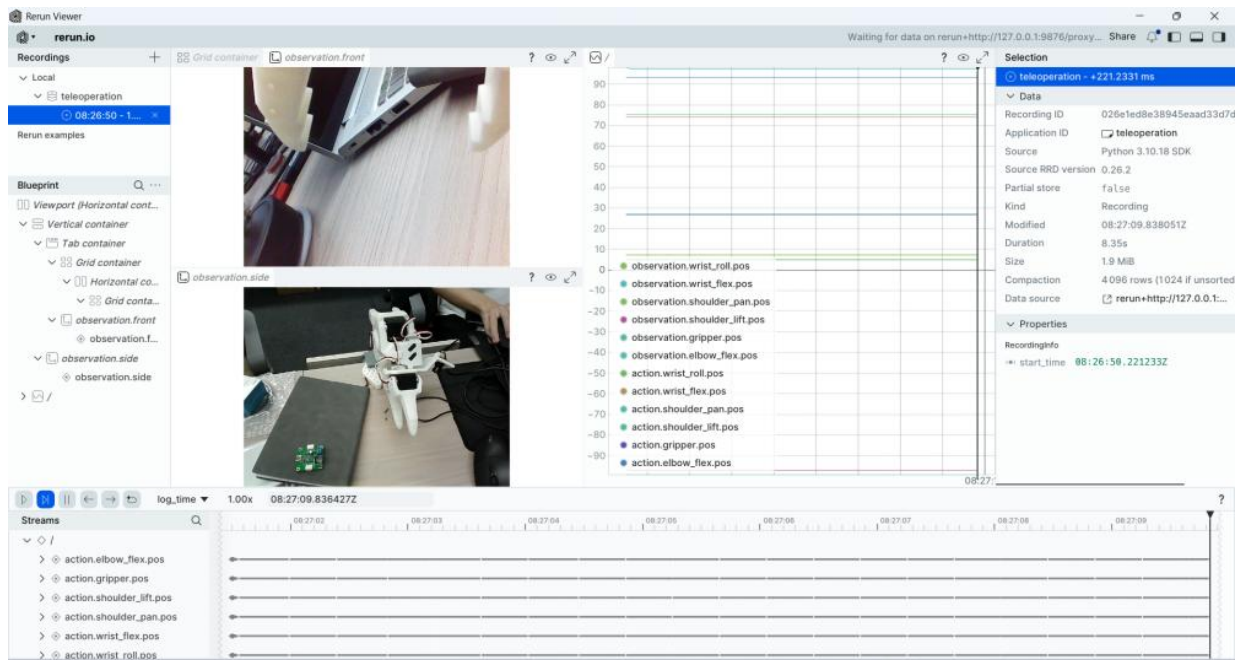
Happy Rerunning!

```

```

C:\Windows\system32\cmd.exe x + v
    'max_relative_target': None,
    'port': 'COM7',
    'use_degrees': False},
'teleop': {'calibration_dir': None,
    'id': 'my_awesome_leader_arm',
    'port': 'COM22',
    'use_degrees': False},
'teleop_time_s': None}
[2025-12-16T08:18:38Z INFO re_grpc_server] Listening for gRPC connections on 0.0.0.0:9876. Connect by running 'rerun -
connect rerun+http://127.0.0.1:9876/proxy'
INFO 2025-12-16 16:18:38 01_leader.py:82 my_awesome_leader_arm S0101Leader connected.
[2025-12-16T08:18:38Z INFO egui_wgpu] There were 2 available wgpu adapters: {backend: Vulkan, device_type: DiscreteGpu,
name: "NVIDIA GeForce RTX 3050 Laptop GPU", driver: "NVIDIA", driver_info: "546.30", vendor: NVIDIA (0x10DE), device: 0
x25E2}, {backend: GL, device_type: Other, name: "NVIDIA GeForce RTX 3050 Laptop GPU/PCIe/SSE2", driver_info: "4.6.0 NVID
IA 546.30", vendor: NVIDIA (0x10DE)}
WARNING 2025-12-16 16:18:39 a_opencv.py:259 OpenCVCamera(1) failed to set fourcc=MJPEG (actual=YUY2, success=True). Cont
nuing with default format.
INFO 2025-12-16 16:18:41 a_opencv.py:180 OpenCVCamera(1) connected.
INFO 2025-12-16 16:18:47 a_opencv.py:180 OpenCVCamera(0) connected.
INFO 2025-12-16 16:18:47 follower.py:104 my_awesome_follower_arm S0101Follower connected.
Teleop loop time: 31.93ms (31 Hz)
-----
NAME | NORM
-----|-----
shoulder_pan.pos | 4.89
shoulder_lift.pos | -99.42
elbow_flex.pos | 93.27
wrist_flex.pos | 74.09
wrist_roll.pos | 7.27
gripper.pos | 26.98

```

7. 数据集采集

7.0 命令简易使用

在源码根目录 (E:\WT\lerobot_xidi\) 编写 record_config.yaml 文件，示例如下

本地数据集采集简化

```
! record_config.yaml
1 # record_config.yaml
2 dataset:
3   root: "D:/MyRobotData" # 请修改为你想要的永久路径
4   dataset.repo_id: WT/test
5   dataset.num_episodes: 20
6   dataset.single_task: "Grab the black cube"
7   dataset.push_to_hub: false
8   dataset.episode_time_s: 60
9   dataset.reset_time_s: 30
10  display_data: true
11  # 你还可以在这里永久设置其他参数，例如：
12  # fps: 30
13  # num_episodes: 20
```

```
lerobot-record --config_path=record_config.yaml --teleop.type=sol101_leader --
teleop.port=COM22 --teleop.id=my_awesome_leader_arm --robot.type=sol101_follower --
robot.port=COM7 --robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv,
```



```
index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, front: {type: opencv,  
index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --  
robot.disable_torque_on_disconnect=true
```

云端数据集采集简化

```
# record_hub_config.yaml  
# 永久配置 S0-101 机械臂主从录制任务  
  
# 1. 遥操作设备（主臂）配置  
teleop:  
  type: sol01_leader  
  port: COM22  
  id: my_awesome_leader_arm  
  
# 2. 机器人（从臂）配置  
robot:  
  type: sol01_follower  
  port: COM7  
  id: my_awesome_follower_arm  
  disable_torque_on_disconnect: true # 安全设置，断开时卸力  
# 摄像头配置（两个摄像头）  
cameras:  
  side:  
    type: opencv  
    index_or_path: 0  
    width: 640  
    height: 480  
    fps: 30  
    fourcc: MJPG  
  front:  
    type: opencv  
    index_or_path: 1  
    width: 640  
    height: 480
```

```

        fps: 30

        fourcc: MJPG

# 3. 数据集录制配置
dataset:
    repo_id: luoxingtong/record_test
    num_episodes: 30
    single_task: "Catch the green oil board"
    push_to_hub: true
    episode_time_s: 60
    reset_time_s: 15

display_data: true
resume: false

lerobot-record --config_path=record_config.yaml --teleop.type=sol101_leader --
teleop.port=COM22 --teleop.id=my_awesome_leader_arm --robot.type=sol101_follower --
robot.port=COM7 --robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, front: {type: opencv,
index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --
robot.disable_torque_on_disconnect=true

```

训练命令更改简化

```

# train_config.yaml

# 1. 数据集配置
dataset:
    repo_id: WT/test
    root: E:/WT/dataset

# 2. ACT 策略配置（核心修正）
policy:
    type: act

# --- 关键修正：当前版本仅支持单步观测 ---

```

```
n_obs_steps: 1          # 从 2 改为 1，以兼容当前 LeRobot 版本
# --- 强制使用 GPU 进行训练 ---
device: cuda           # 显式指定，覆盖默认的 'None'，确保使用 RTX 3050
# --- 模型架构参数 ---
dim_model: 512
n_heads: 8
dim_feedforward: 2048
n_encoder_layers: 8
n_decoder_layers: 8
# --- 序列与动作参数 ---
chunk_size: 100
n_action_steps: 50
# --- 优化与训练 ---
optimizer_lr: 1.0e-4
optimizer_weight_decay: 1.0e-4
use_amp: true          # 启用自动混合精度，可节省显存并加速（RTX 3050 支持）
push_to_hub: false

# 3. 训练流程参数
output_dir: outputs/train/act_sol01_test
job_name: act_sol01_test
num_workers: 2
batch_size: 8          # 如果启用混合精度后仍显存不足，可降低到 4
steps: 300000
eval_freq: 5000
log_freq: 100
save_checkpoint: true
save_freq: 10000
resume: false
seed: 42

# 4. 其他配置
eval:
    n_episodes: 10
    batch_size: 10      # 这行是关键！必须存在且值 <= n_episodes
```

```
wandb:
  enable: true

lerobot-record --config_path=train_config.yaml
```

评估命令简化

```
#eval_config.yaml
# 永久配置 S0-101 机械臂主从评估任务

# 1. 机器人（从臂）配置
robot:
  type: sol01_follower
  port: COM7
  id: my_awesome_follower_arm
  disable_torque_on_disconnect: true # 安全设置，断开时卸力
# 摄像头配置（两个摄像头）
cameras:
  side:
    type: opencv
    index_or_path: 0
    width: 640
    height: 480
    fps: 30
    fourcc: MJPG
  front:
    type: opencv
    index_or_path: 1
    width: 640
    height: 480
    fps: 30
    fourcc: MJPG
# 2. 机器人数据集配置
dataset:
  repo_id: WT/eval_test
```

```
single_task: "Grab the screwdriver"
# 3. 预训练模型配置
policy:
  path: outputs/train/act_sol01_test/checkpoints/020000/pretrained_model

display_data: true

lerobot-record --config_path=eval_config.yaml
```

7.1 数据集采集到本地

运行指令，进行数据采集， `{HF_USER}` 可以改为自己用户名（注意用户名必须为英文）。这里的指令

我们以采集 20 次为例，如果需要采集更多次，需要将 `dataset.num_episodes=20` 这个 20 改为更多次数。

```
lerobot-record --teleop.type=sol01_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm --robot.type=sol01_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, front: {type:
opencv, index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --
dataset.repo_id=WT/test --dataset.num_episodes=20 --dataset.single_task="Grab the
black cube" --dataset.push_to_hub=false --dataset.episode_time_s=60 --
dataset.reset_time_s=30 --display_data=true --
robot.disable_torque_on_disconnect=true
```

其中 `repo_id` 可以自定义修改，`push_to_hub=false` 表示不上传云端，最后数据集会保存在主目录的 `C:/用户/用户名/.cache/huggingface/lerobot` 下会创建上述 `WT/test` 文件夹

7.2 数据集采集到云端

huggingface 设置(云端上传所用, 如要设置请先跳转章节 9 参考教程)

如果您想使用 Hugging Face Hub 的功能来上传您的数据集, 并且您之前尚未这样做, 请确保您已使用具有写入权限的令牌登录, 该令牌可以从 [HuggingFace 登录](#) 中生成(每次退出终端后使用都要运行 `hf auth login` 登录):

```
huggingface-cli login --token ${HUGGINGFACE_TOKEN} --add-to-git-credential
```

```
(lerobot) E:\WT\lerobot_xidi>huggingface-cli login --token [REDACTED] --add-to-git-credential
⚠ Warning: 'huggingface-cli login' is deprecated. Use 'hf auth login' instead.
Token is valid (permission: fineGrained).
The token 'wtwl' has been saved to C:\Users\梦、星、枫\.cache\huggingface\stored_tokens
Your token has been saved in your configured git credential helpers (manager).
Your token has been saved to C:\Users\梦、星、枫\.cache\huggingface\token
Login successful.
The current active token is: 'wtwl'
```

将您的 Hugging Face 仓库名称存储在一个变量中, 以运行以下命令:

```
for /f "delims=" %i in ('hf auth whoami') do set "HF_USER=%i"
```

```
(lerobot) E:\WT\lerobot_xidi>for /f "delims=" %i in ('hf auth whoami') do set "HF_USER=%i"
(lerobot) E:\WT\lerobot_xidi>set "HF_USER=user: luoxingtong" ← 这个是输出不用自己输入
```

记录 5 个回合并将您的数据集上传到 Hub (HF_USER 就是上面输出的 user):

```
lerobot-record --teleop.type=sol101_leader --teleop.port=COM22 --
teleop.id=my_awesome_leader_arm --robot.type=sol101_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv,
index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, front: {type:
opencv, index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --
dataset.repo_id=${HF_USER}/record_test --dataset.num_episodes=5 --
dataset.single_task="Grab the black cube" --dataset.push_to_hub=true --
dataset.episode_time_s=30 --dataset.reset_time_s=30 --
robot.disable_torque_on_disconnect=true
```

7.3 记录功能

`record` 功能提供了一套工具, 用于在机器人运行期间捕获和管理数据。

1. 数据存储

- 数据以 LeRobotDataset 格式存储，并在记录过程中保存到磁盘中。
- 默认情况下，数据集在记录完成后会推送到你的 Hugging Face 页面。
- 若要禁用上传，请使用：`--dataset.push_to_hub=False`。

2. 检查点与恢复

- 在记录过程中会自动创建检查点。
- 如果记录过程中断，可以通过重新运行相同的命令并添加 `--resume=true` 来恢复记录。

重要提示：在恢复时，需将 `--dataset.num_episodes` 设置为要额外记录的剧集数量（而不是数据集中目标的总剧集数量）。

- 若要从头开始记录，请**手动删除**数据集目录。

3. 记录参数

通过命令行参数设置数据记录的流程：

参数	描述	默认值
<code>--dataset.episode_time_s</code>	每个数据剧集的持续时间（秒）	60
<code>--dataset.reset_time_s</code>	每个剧集后环境重置时间（秒）	60
<code>--dataset.num_episodes</code>	要记录的总剧集数量	50

4. 记录过程中的键盘控制

使用键盘快捷键控制数据记录流程：

键	动作
→（右箭头）	提前终止当前剧集/重置；进入下一个。
←（左箭头）	取消当前剧集；重新录制。
ESC	立即停止会话，编码视频，并上传数据集。

```
pip install pynput==1.6.8
```

7.4 数据收集技巧

- **任务建议：** 在不同位置抓取物体并将其放入箱子中。
- **规模：** 记录 ≥ 50 个剧集（每个位置 10 个剧集）。
- **一致性：**
 - 保持摄像头固定。
 - 保持相同的抓取行为。
 - 确保操作的物体在摄像头画面中可见。
- **逐步推进：**
 - 先从可靠的抓取开始，然后再增加变化（新位置、抓取技巧、摄像头调整）。
 - 避免复杂性急剧增加，以防止失败。

经验法则：仅使用摄像头画面作为指导，只根据屏幕反馈的视频图像，来控制机械臂完成任务。

7.5 数据集操作(不稳定，可尝试)

1. 可视化数据集

```
echo ${HF_USER}/sol01_test
```

如果您没有使用 `--dataset.push_to_hub=false`，并上传了数据，您也可以在本机通过以下命令进行可视化：

```
lerobot-dataset-viz --repo-id ${HF_USER}/sol01_test
```

如果您使用了 `--dataset.push_to_hub=false`，没有上传数据，您可以通过以下命令在本机进行可视化：

```
lerobot-dataset-viz --repo-id WT/test
```

这里，WT 是数据收集时自定义的 `repo_id` 名称。

2. 回放数据集

现在，尝试在您的机器人上重播第一个数据集：

```
lerobot-replay --robot.type=sol01_follower --robot.port=COM7 --  
robot.id=my_awesome_follower_arm --dataset.repo_id=${HF_USER}/record-test --  
dataset.episode=0
```

此时，机器人应该做出与你遥操记录时一样的动作。

8. 训练及评估

8.1 pytorch 和 cuda 检查及问题解决

```
python -c "import torch; print('PyTorch 版本:', torch.__version__); print('CUDA 是否可用:', torch.cuda.is_available()); print('CUDA 设备数:', torch.cuda.device_count())"
```

```
(lerobot) E:\WT\lerobot_xidi>python -c "import torch; print('PyTorch版本:', torch.__version__); print('CUDA是否可用:', torch.cuda.is_available()); print('CUDA设备数:', torch.cuda.device_count())"
PyTorch版本: 2.7.1+cpu
CUDA是否可用: False
CUDA设备数: 0
```

改为 gpu 版本

```
nvidia-smi
```

```
(lerobot) E:\WT\lerobot_xidi>nvidia-smi
Wed Dec 17 09:26:25 2025
+-----+
| NVIDIA-SMI 546.30                Driver Version: 546.30      CUDA Version: 12.3     |
+-----+-----+-----+

```

卸载: `pip uninstall torch torchvision torchaudio -y`

```
(lerobot) E:\WT\lerobot_xidi>pip uninstall torch torchvision torchaudio -y
Found existing installation: torch 2.7.1
Uninstalling torch-2.7.1:
  Successfully uninstalled torch-2.7.1
Found existing installation: torchvision 0.22.1
Uninstalling torchvision-0.22.1:
  Successfully uninstalled torchvision-0.22.1
WARNING: Skipping torchaudio as it is not installed.
```

下载稳定版(图片中框选的为 CDUA 最高版本): `pip install torch torchvision torchaudio`

`--index-url https://download.pytorch.org/whl/cu118`

```
Downloading https://download.pytorch.org/whl/cu118/torchaudio-2.7.1%2Bcu118-cp310-cp310-win_amd64.whl (4.1 MB)
4.1/4.1 MB 12.9 MB/s 0:00:00
Installing collected packages: torch, torchvision, torchaudio
Successfully installed torch-2.7.1+cu118 torchaudio-2.7.1+cu118 torchvision-0.22.1+cu118
```

再次运行: `python -c "import torch; print('PyTorch 版本:', torch.__version__); print('CUDA 是否可用:', torch.cuda.is_available()); print('CUDA 设备数:', torch.cuda.device_count())"`

```
(lerobot) E:\WT\lerobot_xidi>python -c "import torch; print('PyTorch版本:', torch.__version__); print('CUDA是否可用:', torch.cuda.is_available()); print('CUDA设备数:', torch.cuda.device_count())"
PyTorch版本: 2.7.1+cu118
CUDA是否可用: True
CUDA设备数: 1
```

8.2 ACT(较为常用)

```
lerobot-train --dataset.repo_id=${HF_USER}/sol101_test --policy.type=act --
output_dir=outputs/train/act_sol101_test --job_name=act_sol101_test --policy.device=cuda --
wandb.enable=false --steps=300000

lerobot-train --dataset.repo_id=WT/test --policy.type=act --
output_dir=outputs/train/act_sol101_test --job_name=act_sol101_test --policy.device=cuda --
wandb.enable=false --policy.push_to_hub=false --steps=300000 --dataset.root=E:\WT\dataset

lerobot-record --robot.type=sol101_follower --robot.port=COM7 --
robot.id=my_awesome_follower_arm --robot.cameras="{ side: {type: opencv, index_or_path: 0,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, front: {type: opencv, index_or_path: 1,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --display_data=true --
dataset.repo_id=WT/eval_test --dataset.single_task="Grab the screwdriver" --
policy.path=outputs/train/act_sol101_test/checkpoints/020000/pretrained_model --
robot.disable_torque_on_disconnect=true
```

8.3 命令解释

数据集指定：我们通过 `--dataset.repo_id=${HF_USER}/sol101_test` 参数提供了数据集。

训练步数：我们通过 `--steps=300000` 修改训练步数，算法默认为 800000，根据自己的任务难易程度，观察训练时候的 loss 来进行调整。

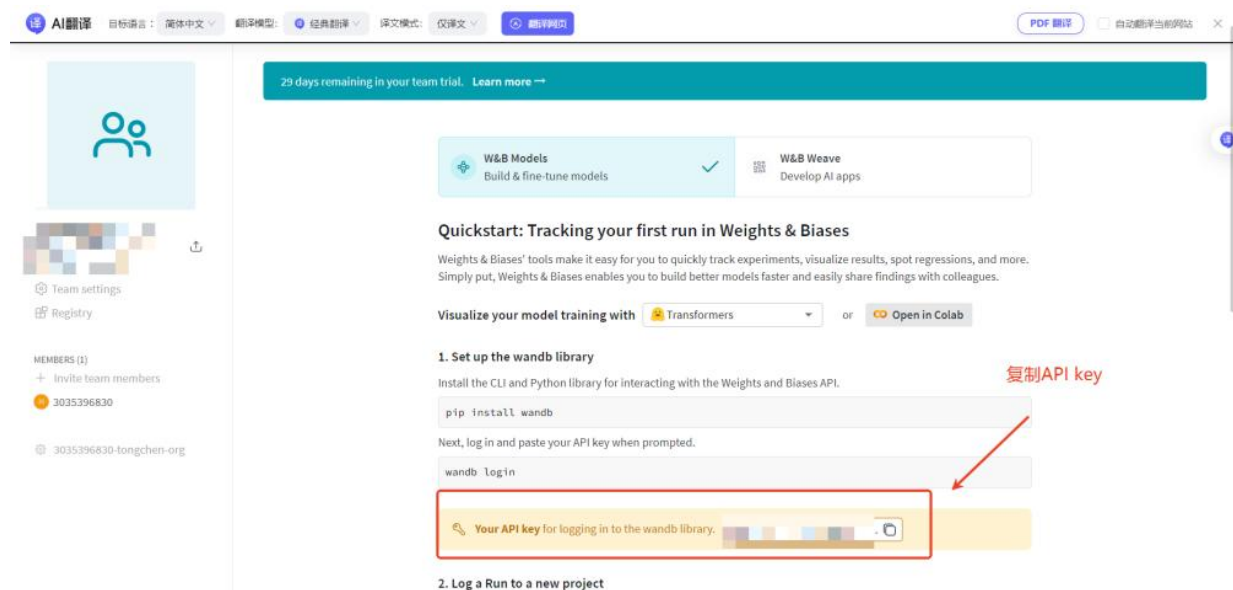
策略类型：我们使用 `policy.type=act` 提供了策略，同样你可以更换 [act, diffusion, pi0, pi0fast, pi0fast, sac, smolvla] 等策略，这将从 `configuration_act.py` 加载配置。重要的是，这个策略会自动适应您机器人（例如 laptop 和 phone）的电机状态、电机动作和摄像头数量，这些信息已保存在您的数据集中。

设备选择：我们提供了 `policy.device=cuda`，因为我们正在 Nvidia GPU 上进行训练，但您可以使用 `policy.device=mps` 在 Apple Silicon 上进行训练。

可视化工具：我们提供了 `wandb.enable=true` 来使用 Weights and Biases 可视化训练图表。这是可选的，但如果您使用它，请确保您已通过运行 `wandb login` 登录。

Weights & Biases

点击后请根据官网提示完成初步注册



输入 `wandb login` 后输入复制的 API key 按 `enter` 确认

```
(lerobot) E:\WT\lerobot_xidi>wandb login
E:\WT\miniconda3\envs\lerobot\lib\site-packages\pydantic\_internal\_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'repr' attribute with value False was provided to the 'Field()' function, which has no effect in the context it was used. 'repr' is field-specific metadata, and can only be attached to a model field using 'Annotated' metadata or by assignment. This may have happened because an 'Annotated' type alias using the 'type' statement was used, or if the 'Field()' function was attached to a single member of a union type.
  warnings.warn(
E:\WT\miniconda3\envs\lerobot\lib\site-packages\pydantic\_internal\_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'frozen' attribute with value True was provided to the 'Field()' function, which has no effect in the context it was used. 'frozen' is field-specific metadata, and can only be attached to a model field using 'Annotated' metadata or by assignment. This may have happened because an 'Annotated' type alias using the 'type' statement was used, or if the 'Field()' function was attached to a single member of a union type.
  warnings.warn(
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit: |
```

```
(lerobot) E:\WT\lerobot_xidi>wandb login
E:\WT\miniconda3\envs\lerobot\lib\site-packages\pydantic\_internal\_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'repr' attribute with value False was provided to the 'Field()' function, which has no effect in the context it was used. 'repr' is field-specific metadata, and can only be attached to a model field using 'Annotated' metadata or by assignment. This may have happened because an 'Annotated' type alias using the 'type' statement was used, or if the 'Field()' function was attached to a single member of a union type.
  warnings.warn(
E:\WT\miniconda3\envs\lerobot\lib\site-packages\pydantic\_internal\_generate_schema.py:2249: UnsupportedFieldAttributeWarning: The 'frozen' attribute with value True was provided to the 'Field()' function, which has no effect in the context it was used. 'frozen' is field-specific metadata, and can only be attached to a model field using 'Annotated' metadata or by assignment. This may have happened because an 'Annotated' type alias using the 'type' statement was used, or if the 'Field()' function was attached to a single member of a union type.
  warnings.warn(
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter, or press ctrl+c to quit:
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: C:\Users\梦、星、枫\netrc
wandb: Currently logged in as: [REDACTED] to https://api.wandb.ai. Use 'wandb login --relogin' to force relogin
```

指令解释

dataset：数据集配置对象，定义训练使用的数据集

env：环境配置对象，用于定义机器人仿真或真实环境的参数（可选）

policy：预训练模型配置，定义策略网络的架构和参数（可选）

output_dir：训练输出目录路径，存储模型检查点、日志等文件。若使用相同路径再次训练会覆盖内容（除非启用 resume）

job_name：任务名称，用于标识当前训练任务（可选）

resume：是否从检查点恢复训练。设为 True 时需确保 output_dir 中存在至少一个检查点文件

seed：随机种子（默认 1000），用于控制模型初始化、数据集打乱和评估环境的随机性，确保实验可复现

num_workers：数据加载器的工作进程数（默认 4），影响数据加载速度

batch_size：批次大小（默认 8），每次迭代处理的样本数量

steps：总训练步数（默认 100,000）

eval_freq：评估频率（默认每 20,000 步），定期在验证集上评估模型性能

log_freq：日志记录频率（默认每 200 步），控制训练指标的记录间隔

save_checkpoint : 是否保存训练检查点（默认 True）

save_freq : 检查点保存频率（默认每 20,000 步），最后一步训练后也会保存

use_policy_training_preset : 是否使用策略训练预配置（默认 True）

optimizer : 优化器配置对象，定义学习率、权重衰减等参数（可选）

scheduler : 学习率调度器配置，定义学习率衰减策略（可选）

eval : 评估配置对象，包含评估相关的详细设置

wandb : Weights & Biases 配置对象，用于实验跟踪和可视化

8.4 SmolVLA

```
pip install -e ".[smolvla]"

lerobot-train --policy.path=lerobot/smolvla_base --dataset.repo_id=${HF_USER}/mydataset --
batch_size=64 --steps=20000 --output_dir=outputs/train/my_smolvla --
job_name=my_smolvla_training --policy.device=cuda --wandb.enable=true

lerobot-record --robot.type=sol01_follower --robot.port=COM7 --
robot.id=my_blue_follower_arm --robot.cameras="{ front: {type: opencv, index_or_path: 1,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, side: {type: opencv, index_or_path: 0,
width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --dataset.single_task="Put lego brick
into the transparent box" --dataset.repo_id=WT/eval_test123 --dataset.episode_time_s=50 --
dataset.num_episodes=10 --policy.path=HF_USER/FINETUNE_MODEL_NAME
```

8.5 Pi0

```
pip install -e ".[pi]"
```



```

lerobot-train --policy.type=pi0 --dataset.repo_id=WT/test123 --job_name=pi0_training --
output_dir=outputs/pi0_training --policy.pretrained_path=lerobot/pi0_base --
policy.compile_model=true --policy.gradient_checkpointing=true --policy.dtype=bfloat16 --
steps=20000 --policy.device=cuda --batch_size=32 --wandb.enable=false --
dataset.root=E:\WT\dataset

lerobot-record --robot.type=sol01_follower --robot.port=COM7 --robot.cameras="{ front:
{type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, side:
{type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --
robot.id=my_awesome_follower_arm --display_data=false --dataset.repo_id=WT/eval_test123 --
dataset.single_task="Put lego brick into the transparent box" --
policy.path=outputs/pi0_training/checkpoints/last/pretrained_model

```

8.6Pi0.5

```

pip install -e ".[pi]"

lerobot-train --dataset.repo_id=WT/test --policy.type=pi05 --
output_dir=outputs/pi05_training --job_name=pi05_training --
policy.pretrained_path=lerobot/pi05_base --policy.compile_model=true --
policy.gradient_checkpointing=true --wandb.enable=false --policy.dtype=bfloat16 --
steps=3000 --policy.device=cuda --batch_size=32 --dataset.root=E:\WT\dataset

lerobot-record --robot.type=sol01_follower --robot.port=COM7 --robot.cameras="{ front:
{type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}, side:
{type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30, fourcc: 'MJPG'}}" --
robot.id=my_awesome_follower_arm --display_data=false --dataset.repo_id=WT/eval_test123 --
dataset.single_task="Put lego brick into the transparent box" --
policy.path=outputs/pi05_training/checkpoints/last/pretrained_model

```

8.7 租赁云训练

8.7.1 智星云

The screenshot displays the ZhiXing Cloud (智星云) interface for renting cloud training resources. The top navigation bar includes links for 算力市场 (Compute Market), 控制台 (Control Panel), 使用文档 (Usage Docs), AI模型库 (AI Model Library), 软件下载 (Software Downloads), 积分商城 (Points Mall), 开放API (Open API), and 硬件市场 (Hardware Market). A 注册/登录 (Register/Login) button is in the top right.

The main content area shows three GPU rental options:

- GeForce RTX 4090 (24G)**: Price ¥1.32. GPU数量: 1 GPU, CPU内存: 8核/16G, 租期模式: 小时租 / 1小时. Buttons: 1GPU机器已满, 预约.
- Tesla V100-32GB (32G)**: Price ¥1.00. GPU数量: 1 GPU, CPU内存: 8核/16G, 租期模式: 小时租 / 1小时. Button: 立即租用.
- GeForce GTX 1080 Ti (11G)**: Price ¥0.70. GPU数量: 1 GPU, CPU内存: 8核/16G, 租期模式: 小时租 / 1小时. Buttons: 1GPU机器已满, 预约.

The bottom section shows the configuration details for a selected instance:

- 自动续租**: 关闭 (Closed). Other options: 按小时续租, 按天续租, 按周续租, 按月续租.
- 磁盘容量**: 系统盘容量: 200G. 添加数据盘 登陆后可选择.
- 保留磁盘**: ☐ 租用结束后删除全部磁盘. ☒ 租用结束后保留全部磁盘 (结束租用后保留磁盘持续计费, 计费价格: 0.0004元/GB每小时, 余额不足时自动释放).
- 当前选择带宽速率**: 32Mbps. A slider shows options from 32Mbps to 500Mbps.
- 系统镜像**: 公共镜像 (Public Image) selected. Options: windows10, ubuntu, centos, deepin.
- 优惠券**: 请选择 (Please select).
- 费用信息**: 预付费: ¥5.70 费用明细. 账户金额: ¥0.00 去充值. 算力券: ¥0.00 去购买.
- 创建实例** button.
- ☐ 优先使用算力券支付.

On the right side, there are links for 联系客服 (Contact Customer Service), 服务号 (Service Number), and 订阅号 (Subscription Number).

实例名	状态 (默认)	保留磁盘	自动续租	开始时间	到期时间	结束时间	机器类型	CPU/内存	系统盘/数据盘	操作
19305396430_lvg1070_54f2948... GeForce RTX 4090 *1 备注: 🔗	运行中 🔗	☐ 到期保留磁盘 ☒ 到期删除磁盘	未开启	2025-09-10 09:42:28	2025-09-10 11:42:28	尚未结束	云主机	8核 16G	200G —	免费: 查看连接方式 更多

连接方式

19305396430_lvg1070_54f2948a3e4848bb90d5c4427312dc2f_u1_c0



VNC

SSH连接

预留端口

SSH地址: js2.blockelite.cn [🔗](#) 端口: 17024 [🔗](#)

SSH账户: root [🔗](#) 密码: [🔗](#)

SSH账户: vipuser [🔗](#) 密码: [🔗](#)

SSH登录命令: ssh root@js2.blockelite.cn -p 17024 [🔗](#)

[webSSH一键登录](#)

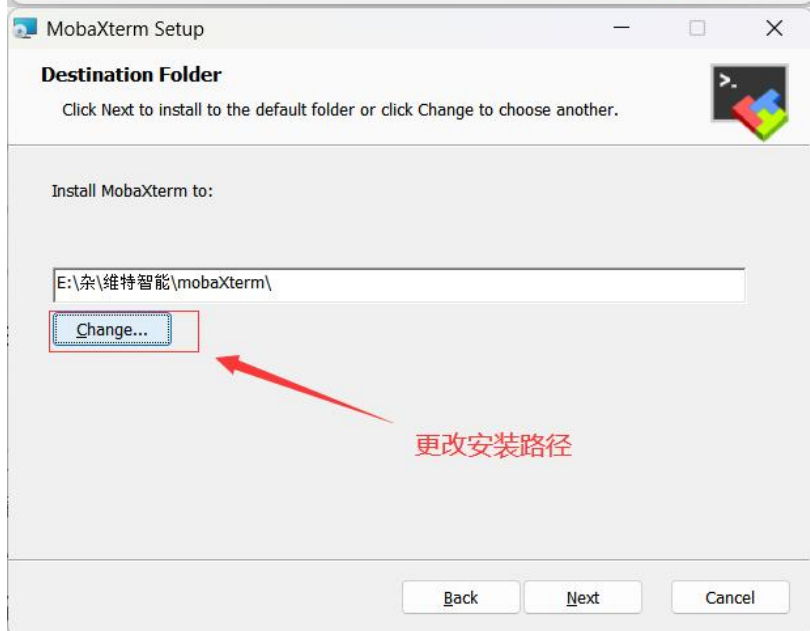
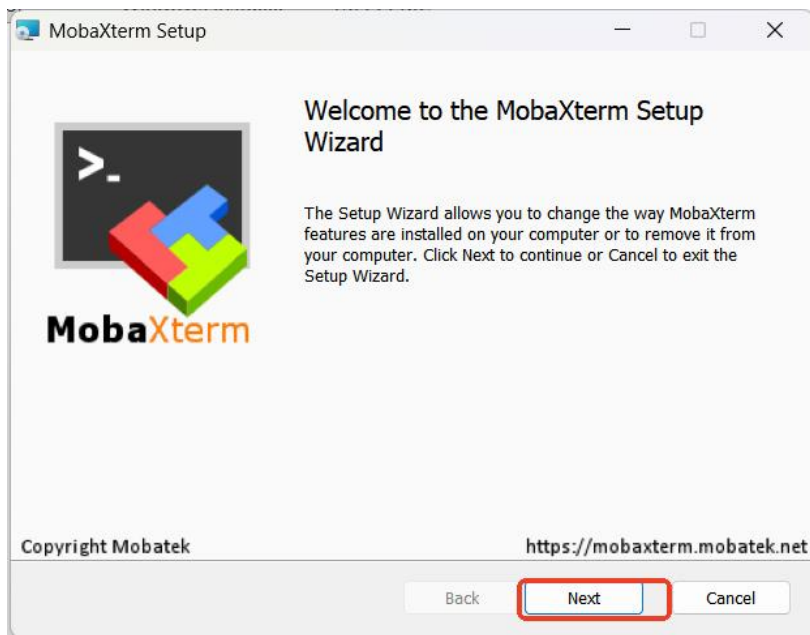
确定

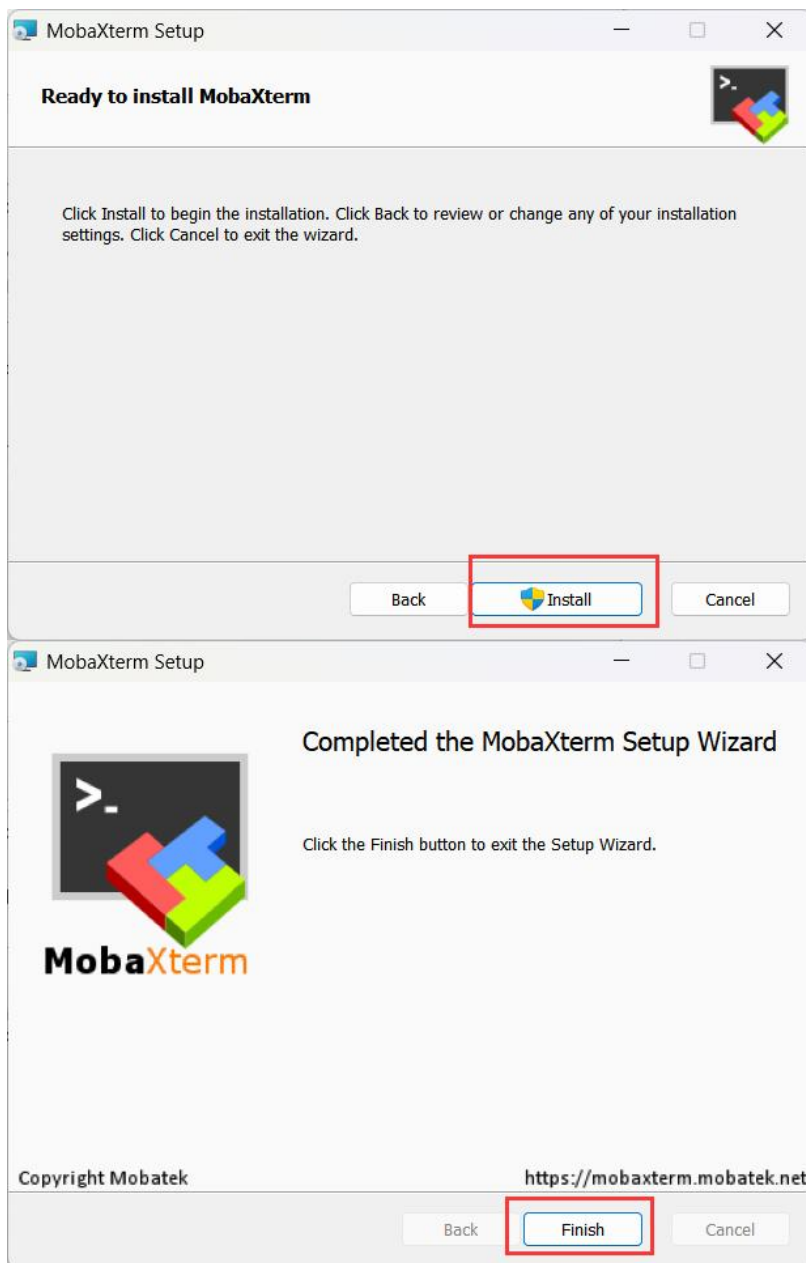
8.7.2 数据集上传

MobaXterm 安装

解压该文件并进入解压后的文件夹，双击应用安装程序

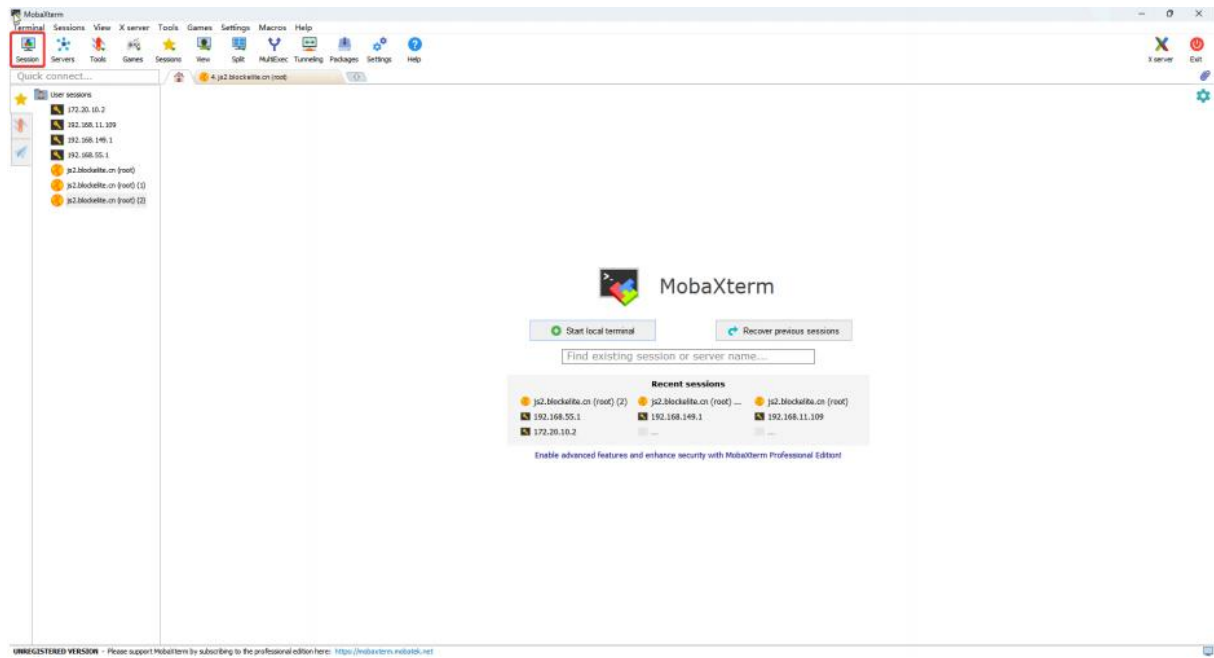
MobaXterm_Installer_v22.1	2025/12/2 18:11	zip file	27,332 KB
MobaXterm_installer.dat	2022/6/5 22:32	DAT 文件	15,649 KB
MobaXterm_installer_22.1	2022/6/5 22:32	Windows Install...	13,444 KB

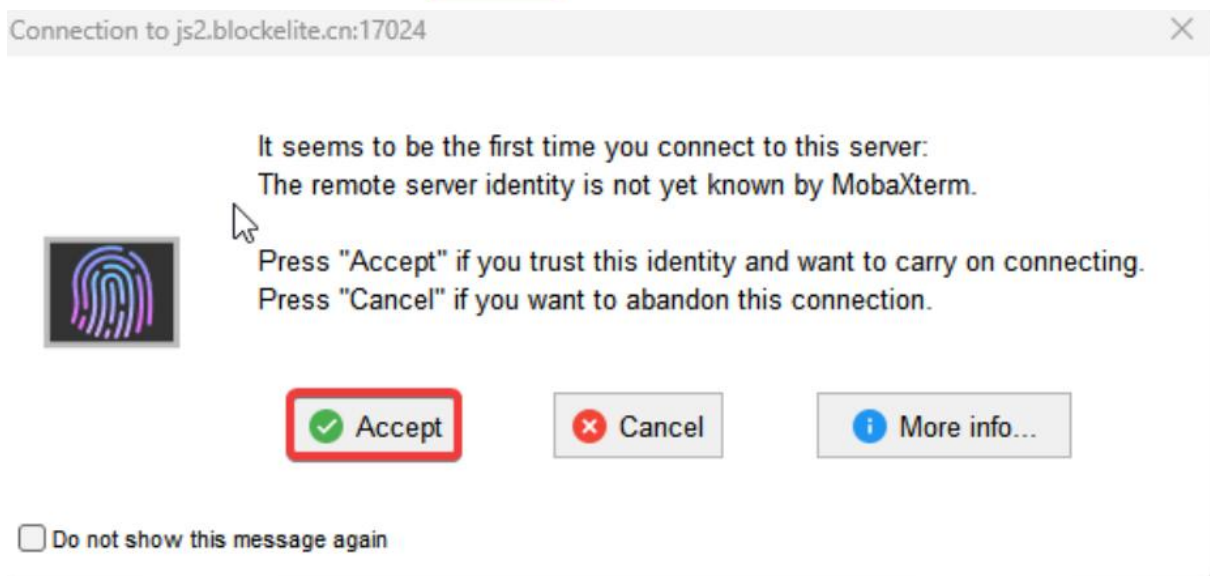
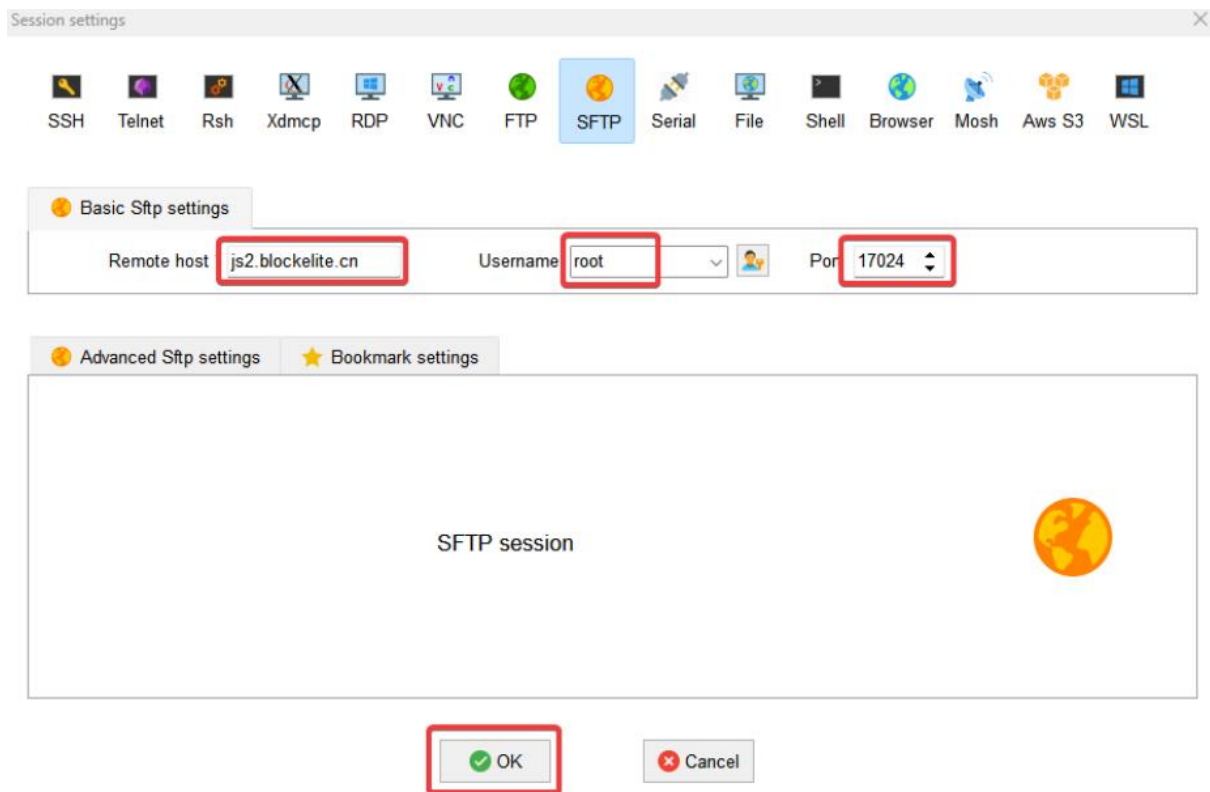




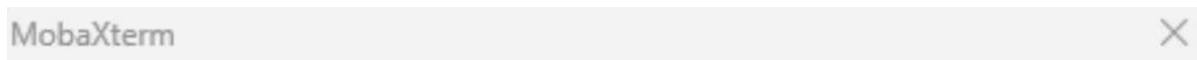
MobaXterm 连接智星云服务器

使用 SFTP 进行文件传输，将租用云主机的地址、用户名和端口进行填写，如下图所示，然后点击 OK。





输入云主机的密码，进行登录。



Please enter your password for root on js2.blockelite.cn

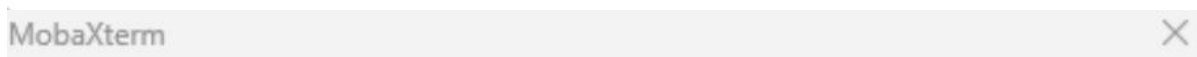
☐ Show password



OK



Cancel



Do you want to save password for root@js2.blockelite.cn?



Yes



No

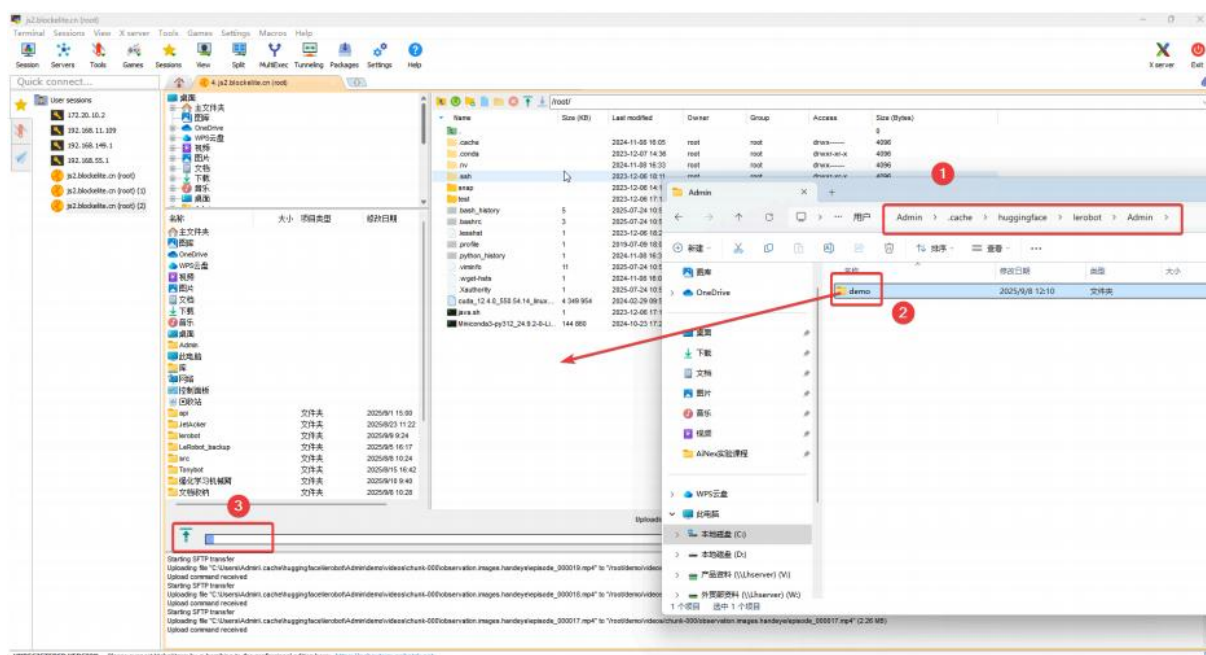
MobaXterm will use your "Master Password" in order to secure all your stored passwords with strong encryption.

☐ Do not show this message again



打开文件夹，找到采集的数据，这里存放的路径为

C:\Users\Admin\.cache\huggingface\lerobot\Admin，将 demo 文件夹整个拖到云主机上。

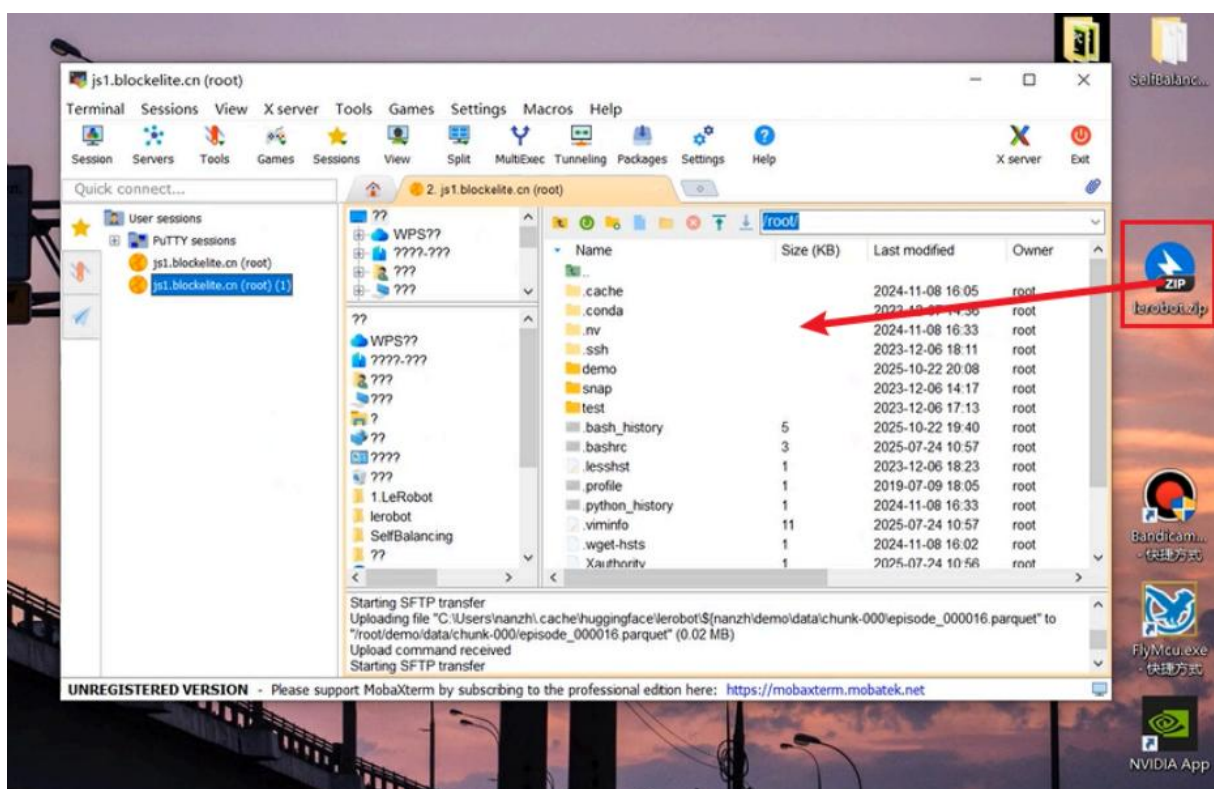


传输完成后，可查看 root 目录下是否有一个 demo 文件夹。

/root/						
Name	Size (KB)	Last modified	Owner	Group	Access	Size (Bytes)
.						0
.cache		2024-11-08 16:05	root	root	drwx-----	4096
.conda		2023-12-07 14:36	root	root	drwxr-xr-x	4096
.nv		2024-11-08 16:33	root	root	drwx-----	4096
.ssh		2023-12-06 18:11	root	root	drwxr-xr-x	4096
demo		2025-09-10 09:58	root	root	drwxr-xr-x	4096
snap		2023-12-06 14:17	root	root	drwx-----	4096
test		2023-12-06 17:13	root	root	drwxr-xr-x	4096
.bash_history	5	2025-07-24 10:57	root	root	-rw-----	5318
.bashrc	3	2025-07-24 10:57	root	root	-rw-r--r--	3869
.lessht	1	2023-12-06 18:23	root	root	-rw-----	20
.profile	1	2019-07-09 18:05	root	root	-rw-r--r--	161
.python_history	1	2024-11-08 16:33	root	root	-rw-----	85
.viminfo	11	2025-07-24 10:57	root	root	-rw-----	12162
.wget-hsts	1	2024-11-08 16:02	root	root	-rw-r--r--	183
.Xauthority	1	2025-07-24 10:56	root	root	-rw-----	108
cuda_12.4.0_550.54.14_linux....	4 349 954	2024-02-29 09:51	root	root	-rw-r--r--	4454353277
java.sh	1	2023-12-06 17:19	root	root	-rw-r--r--	833
Miniconda3-py312_24.9.2-0-Li...	144 860	2024-10-23 17:21	root	root	-rw-r--r--	148337011

将 lerobot.zip 文件拖到云主机上。

传输完成后，可查看 root 目录下是否有一个 lerobot.zip 文件。



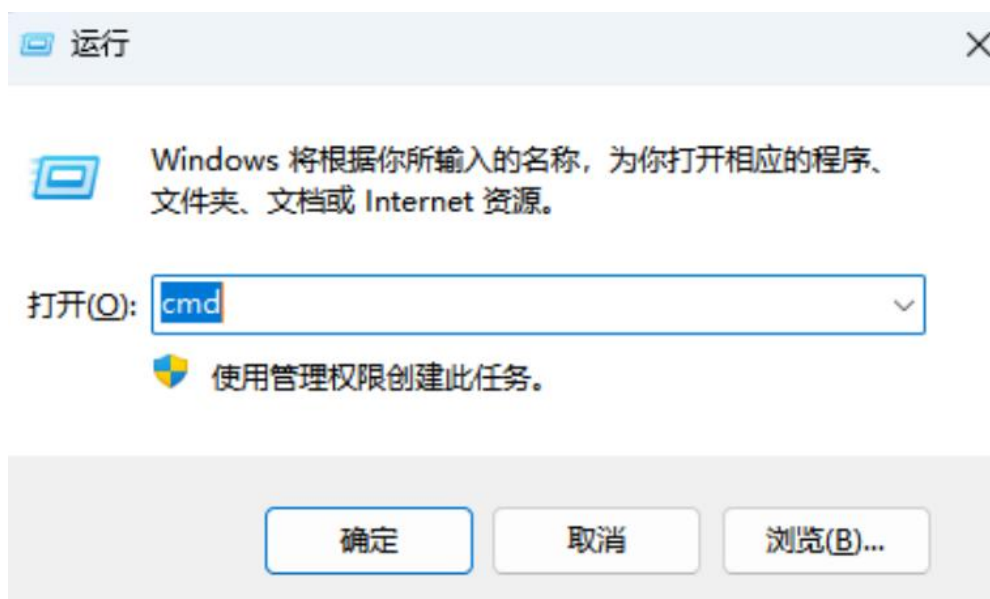
8.7.3 数据集训练

云端主机配置训练

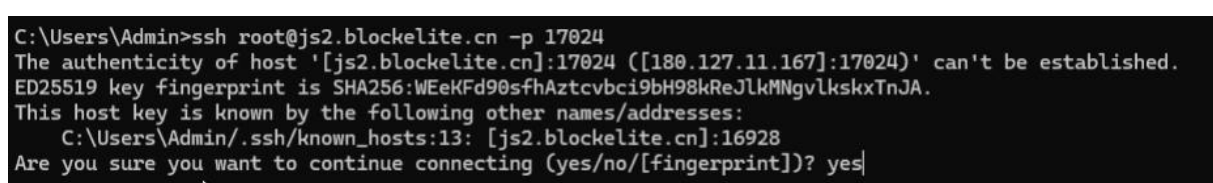
点击复制 SSH 登录命令。



win+R 打开控制面板，输入 cmd，打开终端。



将指令复制到终端中，点击 Enter，输入 yes 后点击 Enter。



输入云主机密码。

```
root@js2.blockelite.cn's password:
```

下图所示则为登录成功。

```
(base) root@ubuntu22:~# |
```

使用 apt 工具下载 screen。

```
apt install -y
```

```
screen
```

```
(base) root@ubuntu22:~# apt install -y screen
```

使用 unzip 工具解压 lerobot.zip 文件。

```
unzip
```

```
(base) root@ubuntu22:~# unzip lerobot.zip | 使
```

用 Miniconda 创建虚拟环境并安装 ffmpeg。

```
conda create -n lerobot python=3.10.18 ffmpeg=7.1.1 -c conda-
```

```
forge
```

```
(base) root@ubuntu22:~# conda create -n lerobot python=3.10
```

出现下图所示的提示，则说明创建成功。

```
# To activate this environment, use
#
#     $ conda activate lerobot
#
# To deactivate an active environment, use
#
#     $ conda deactivate
```

活虚拟环境。

```
conda activate
```

```
lerobot
```

```
(base) root@ubuntu22:~# conda activate lerobot  
(lerobot) root@ubuntu22:~# |
```

使用 `cd` 工具，跳转到 `lerobot` 目录下，安装 `lerobot` 的依赖包并且指定添加飞特舵机相关的驱动。

```
cd lerobot
```

```
pip install -e ".[feetech]" -i  
https://pypi.tuna.tsinghua.edu.cn/simple
```

```
(lerobot) root@ubuntu22:~/lerobot# pip install -e ".[feetech]"
```

 输入指令，创建数据集存储路径。

```
mkdir -p
```

```
/root/.cache/huggingface/lerobot/
```

```
(lerobot) root@ubuntu22:~/lerobot# mkdir -p /root/.cache/huggingface/lerobot/
```

 将

`/root/demo` 移到 `/root/.cache/huggingface/lerobot` 下。

```
mv /root/demo/
```

```
/root/.cache/huggingface/lerobot/
```

```
(lerobot) root@ubuntu22:~/lerobot# mv /root/demo/ /root/.cache/huggingface/lerobot/
```

创建一个名为 `lerobot` 的 session，如需退出 session，`Ctrl+a+d` 即可退出 session（`screen` 是一个终端

复用器，它允许你在一个终端窗口中创建多个虚拟终端会话）。

```
screen -S
```

```
lerobot
```

```
(lerobot) root@ubuntu22:~/lerobot# screen -S lerobot
```

输入指令，即可进入 session。

```
screen -r lerobot
```

激活虚拟环境。

```
conda activate lerobot
```

```
(base) root@ubuntu22:~# conda activate lerobot  
(lerobot) root@ubuntu22:~#
```

使用 `cd` 指令进行跳转目录，输入指令，跳转到 `~/lerobot/src` 下。

```
(lerobot) root@ubuntu22:~/lerobot# cd ~/lerobot/src/
```

输入指令，进行训练。

```
python lerobot/scripts/train.py --dataset.repo_id=demo --policy.type=act --
```

```
output_dir=outputs/train/act_sol101_test --job_name=act_sol101_test --
```

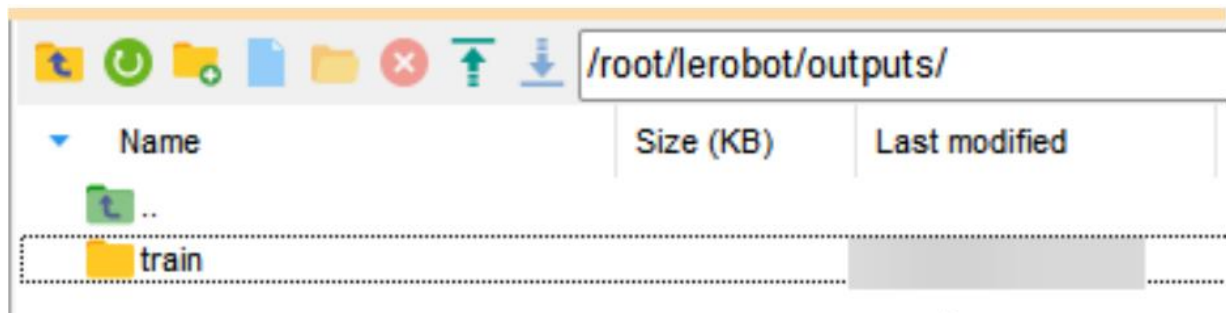
```
policy.device=cuda --wandb.enable=false --
```

```
policy.push_to_hub=false
```

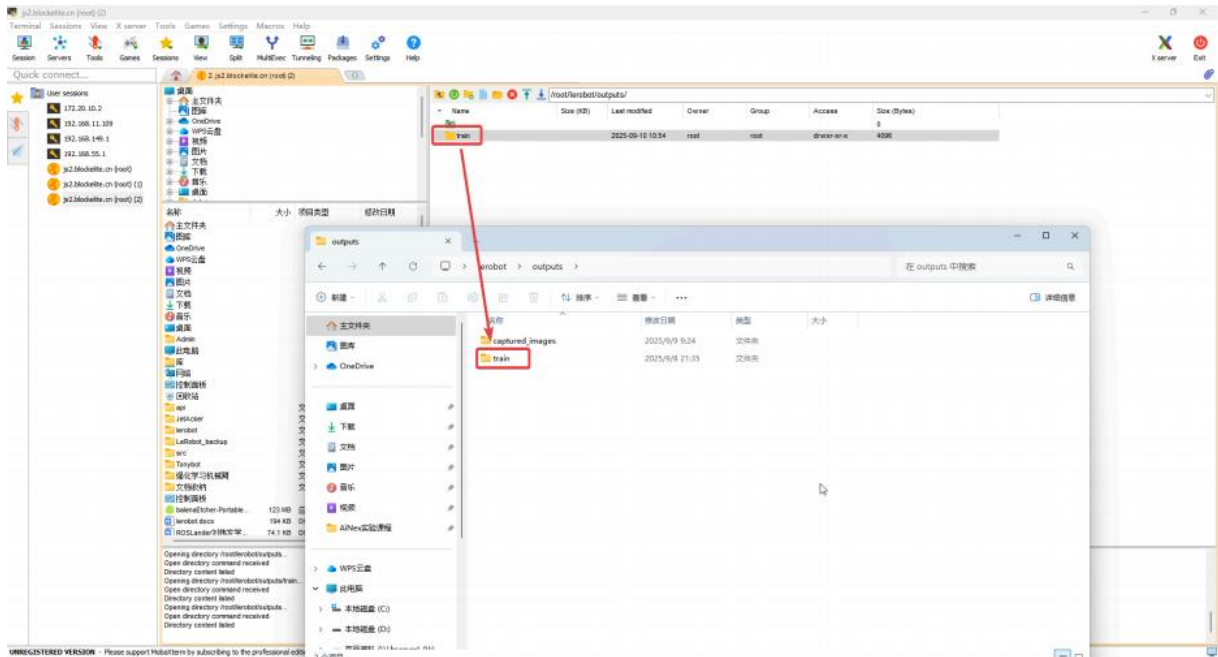
```
(lerobot) root@ubuntu22:~/lerobot/src# python lerobot/scripts/train.py --dataset.repo_id=demo --policy.type=act --output_dir=outputs/train/act_sol101_test --job_name=act_sol101_test --policy.device=cuda --wandb.enable=false --policy.push_to_hub=false
```

训练完，将在 `lerobot/outputs` 生成一个 `train` 的文件夹，具体路径

为 `/root/lerobot/outputs/train` 。



继续使用 SFTP，将训练好的文件传输到本机上的工程文件下，路径为：`lerobot\outputs` 。



至此就完成训练步骤了，接下来就可以推理测试了。

8.7 评估

您可以使用 `lerobot/record.py` 中的 `record` 功能，但需要将策略训练结果训练结果权重文件作为输入。例如，运行以下命令记录 10 个评估回合：

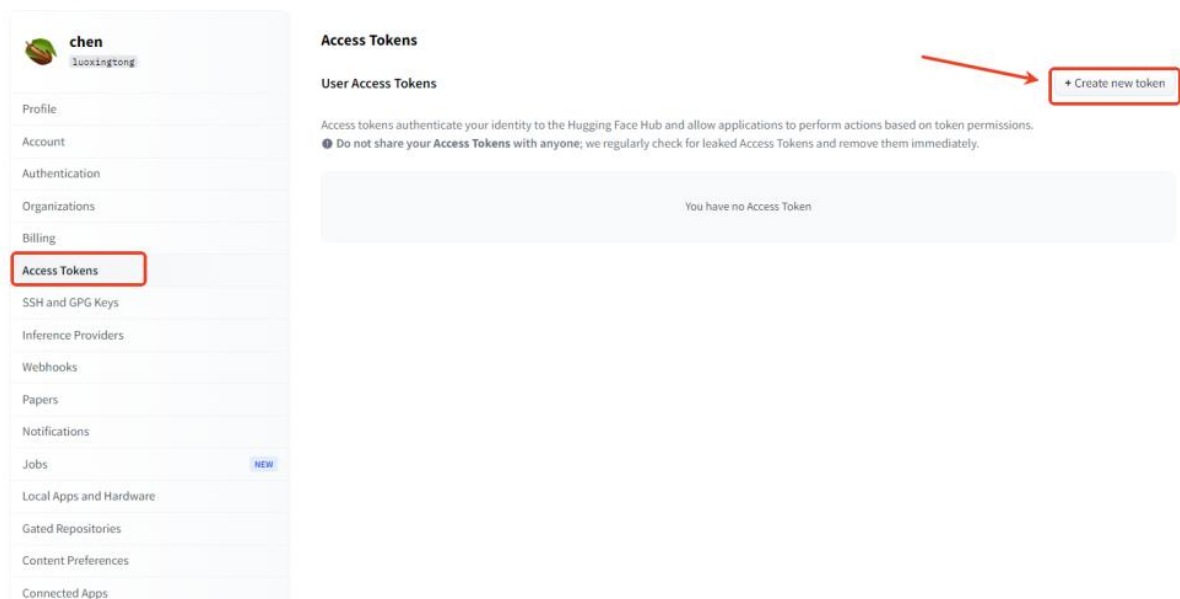
```
lerobot-record --robot.type=sol01_follower --robot.port=/dev/ttyACM0 --
robot.cameras="{ front: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30,
fourcc: 'MJPG'}, side: {type: opencv, index_or_path: 0, width: 640, height: 480, fps:
30, fourcc: 'MJPG'}}" --robot.id=my_awesome_follower_arm --display_data=true --
dataset.repo_id=WT/eval_test --dataset.single_task="Put lego brick into the transparent
box" --policy.path=outputs/train/act_sol01_test/checkpoints/last/pretrained_model
```

1. `--policy.path` 参数，指示您的策略训练结果权重文件的路径（例如 `outputs/train/act_sol01_test/checkpoints/last/pretrained_model`）。如果您将模型训练结果权重文件上传到 Hub，也可以使用模型仓库（例如 `${HF_USER}/act_sol100_test`）。

- 数据集的名称 `dataset.repo_id` 以 `eval_` 开头，这个操作会在你评估的时候为你单独录制评估时候的视频和数据，将保存在 `eval_` 开头的文件夹下，例如 `seed/eval_test123`。

9.hugging face 令牌申请

注册好后会跳到如下界面，请在最上方申请邮箱验证后执行以下步骤添加令牌



如果没有出现该界面请按照如下图片执行

Enterprise Pricing  

Profile

 **luoxingtong**

• Notifications

Inbox (0)

+ New Model

+ New Dataset

+ New Space

+ New Collection

Create organization

Usage Quota

Private Storage 0 GB/100 GB

Zero GPU 0/4 min

Inference Usage \$0.00 / \$0.10

Get Hugging Face PRO →

Settings

Access Tokens

Billing

Changelog

Sign Out

chen
luoxingtong

Profile

Account

Authentication

Organizations

Billing

Access Tokens

SSH and GPG Keys

Inference Providers

Webhooks

Papers

Notifications

Jobs

Local Apps and Hardware

Gated Repositories

Content Preferences

Connected Apps

MCP

Theme

Create new Access Token

Token type

Fine-grained

Read

Write

This cannot be changed after token creation.

Token name

wtwl

User permissions (luoxingtong)

Repositories

☒ Read access to contents of all repos under your personal namespace

☒ Read access to contents of all public gated repos you can access

☒ Write access to contents/settings of all repos under your personal namespace

Webhooks

☐ Access webhooks data

☐ Create and manage webhooks

Discussions & Posts

☐ Interact with discussions / Open PRs on repos under your personal namespace

☐ Interact with discussions / Open PRs on external repos

☐ Interact with posts

Inference

☒ Make calls to Inference Providers

☒ Make calls to your Inference Endpoints

☒ Manage your Inference Endpoints

Scope to specific endpoint(s)

Collections

☐ Read access to all collections under your personal namespace

☐ Write access to all collections under your personal namespace

Billing

☐ Read access to your billing usage and know if a payment method is set

Jobs

☐ Start and manage Jobs on your behalf

Repositories permissions

Override any user-level or org-level permissions set below for the specified repositories. The token will always have read access to all public repos contents.

Search for repos

☐ Read access to contents of selected repos

☐ Interact with discussions / Open pull requests on selected repos

☐ Write access to contents/settings of selected repos

Org permissions

None if not specified.

Search for orgs

Repositories

☐ Read access to contents of all repos in selected organizations

☐ Interact with discussions / Open pull requests on repos in selected organizations

☐ Write access to contents/settings of all repos in selected organizations

Org settings

☐ Read access to organizations settings

☐ Write access to organizations settings / member management

Resource Groups

☐ Write access to resource groups in selected organizations

Inference

☐ Make calls to Inference Providers on behalf of the selected organizations

☐ Make calls to the organization's Inference Endpoints

☐ Manage the organization's Inference Endpoints

Collections

☐ Read access to all collections in selected organizations

☐ Write access to all collections in selected organizations

Jobs

☐ Start and manage Jobs in selected organizations.

Save token

Save your Access Token

Save your token value somewhere safe. You will not be able to see it again after you close this modal. If you lose it, you'll have to create a new one.

复制token

Copy

Name

wtwl

Permissions

FINEGRAINED

Done

Hugging Face

Search models, datasets, users...

Models

Datasets

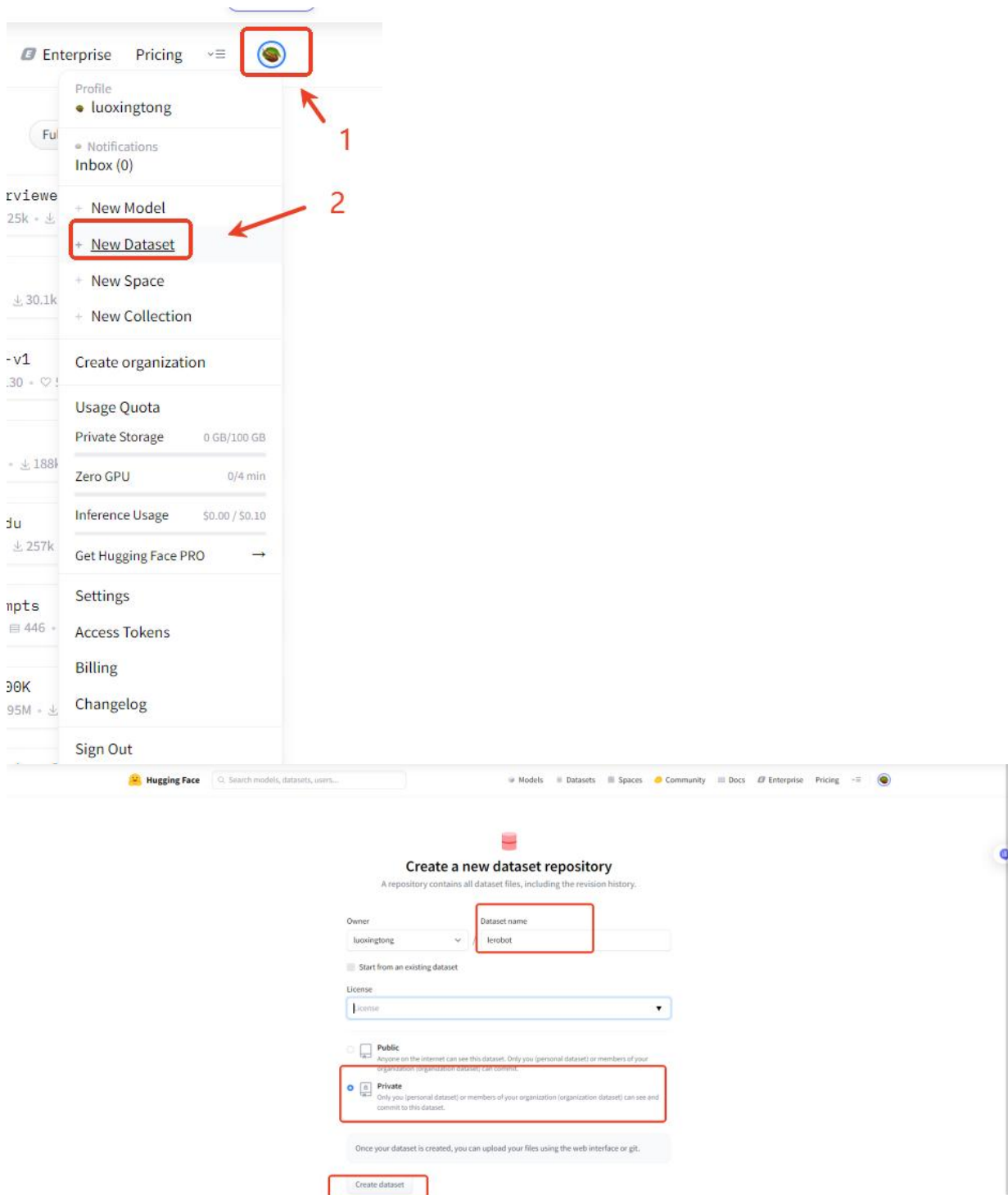
Spaces

Community

Docs

Enterprise

Pricing



10. 报错汇总

出现如下报错请拔出对应舵机连接的舵机线再插入(注意看舵机非齿轮端的 LED 灯是否常亮, 闪烁则电压不足, 不亮则表示电源未接上)

```

Missing motor IDs:
- 5 (expected model: 777)
- 6 (expected model: 777)

Full expected motor list (id: model_number):
{1: 777, 2: 777, 3: 777, 4: 777, 5: 777, 6: 777}

Full found motor list (id: model_number):
{1: 777, 2: 777, 3: 777, 4: 777}

```

ConnectionError: Failed to sync read 'Present_Position' on ids=[1,2,3,4,5,6] after 1 tries. [TxRxResult] There is no status packet!

需要检查对应端口号的机械臂是否接通电源，总线舵机是否出现数据线松动或者脱落，哪个舵机灯不亮就是前面那个舵机的线松了

Magnitude 30841 exceeds 2047 (max for sign_bit_index=11)

对机械臂进行重新断电和上电，再次尝试校准机械臂加准，如果在校准过程中遇到 MAX 角度达到上万的值也可以使用这个方法，如果不行则需要对相应舵机进行重新舵机校准，即中位校准和 ID 写入。

出现如下报错，请修改 camera_opencv.py (兼容性问题)

```

name: "NVIDIA GeForce RTX 3050 Laptop GPU", driver: "NVIDIA", driver_info: "546.30", vendor: NVIDIA (0x10DE), device: 0
x25E2}, {backend: GL, device_type: Other, name: "NVIDIA GeForce RTX 3050 Laptop GPU/PCIe/SSE2", driver_info: "4.6.0 NVID
IA 546.30", vendor: NVIDIA (0x10DE)}
INFO 2025-12-16 15:13:58 01_leader.py:82 my_awesome_leader_arm S0101Leader connected.
Traceback (most recent call last):
  File "E:\WT\miniconda3\envs\lerobot\lib\runpy.py", line 196, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "E:\WT\miniconda3\envs\lerobot\lib\runpy.py", line 86, in _run_code
    exec(code, run_globals)
  File "E:\WT\miniconda3\envs\lerobot\Scripts\lerobot-teleoperate.exe\__main__.py", line 6, in <module>
  File "E:\WT\lerobot_xidi\src\lerobot\scripts\lerobot-teleoperate.py", line 221, in main
    teleoperate()
  File "E:\WT\lerobot_xidi\src\lerobot\configs\parser.py", line 233, in wrapper_inner
    response = fn(cfg, *args, **kwargs)
  File "E:\WT\lerobot_xidi\src\lerobot\scripts\lerobot-teleoperate.py", line 197, in teleoperate
    robot.connect()
  File "E:\WT\lerobot_xidi\src\lerobot\robots\s0101_follower\s0101_follower.py", line 101, in connect
    cam.connect()
  File "E:\WT\lerobot_xidi\src\lerobot\cameras\opencv\camera_opencv.py", line 168, in connect
    raise ConnectionError(
ConnectionError: Failed to open OpenCVCamera(0).Run 'lerobot-find-cameras opencv' to find available cameras.

```

出现如下问题输入 `set PYTHONUTF8=1`

```
UnicodeDecodeError: 'gbk' codec can't decode byte 0xaf in position 91: illegal multibyte sequence
```

File exists: 'home/xxxx/.cache/huggingface/lerobot/xxxxx/seed/eval_xxxx'

请先删除 eval_开头的这个文件夹再次运行程序

`mean` is infinity. You should either initialize with `stats` as an argument or use a pretrained model

请注意--robot.cameras 这个参数中的 front 和 side 等关键词必须和采集数据集的时候保持严格一致。

出现如图报错，是因为 hugging face 仓库没有对应 info.json 等标准配置文件和版本标签

```
File "E:\WT\lerobot_xidi\src\lerobot\datasets\utils.py", line 532, in get_safe_version
    raise RevisionNotFoundError(
huggingface_hub.errors.RevisionNotFoundError: Your dataset must be tagged with a codebase version.
    Assuming _version_ is the codebase_version value in the info.json, you can run this:
    ```python
 from huggingface_hub import HfApi

 hub_api = HfApi()
 hub_api.create_tag("luoxingtong/record_test", tag="_version_", repo_type="dataset")
    ```
```

解决方法如下：

```
resume: true -> resume: false
```

出现视频并没有按照序号分开录制，而是小于应出现的视频数量

首先安装需要的依赖

```
pip install ffmpeg-python pyyaml
```

运行视频分割脚本（如果有原始视频）

```
python fix_video_segmentation.py
```

```
# fix_video_segmentation.py

import json
import os
import subprocess
import shutil
from pathlib import Path
import ffmpeg
import numpy as np
```

```
def split_video_by_episodes(dataset_path):  
    """将单个视频文件分割为每个 episode 独立的视频文件"""  
    dataset_path = Path(dataset_path)  
  
    # 读取元数据  
    meta_path = dataset_path / "meta" / "info.json"  
    with open(meta_path, 'r', encoding='utf-8') as f:  
        info = json.load(f)  
  
    # 读取 episode_data  
    episode_data_path = dataset_path / "meta" / "episode_data.parquet"  
    import pandas as pd  
    episode_df = pd.read_parquet(episode_data_path)  
  
    # 视频文件路径  
    video_dir = dataset_path / "videos" / "observation.images.side" / "chunk-000"  
    input_video = video_dir / "file-000.mp4"  
  
    if not input_video.exists():  
        print(f"视频文件不存在: {input_video}")  
        return False  
  
    # 获取视频信息  
    try:  
        probe = ffmpeg.probe(str(input_video))  
        video_info = next(s for s in probe['streams'] if s['codec_type'] == 'video')  
        fps = eval(video_info['r']) # 帧率  
        duration = float(video_info['duration']) # 总时长  
        total_frames = int(float(video_info.get('nb_frames', duration * fps)))  
    except:  
        print("无法获取视频信息, 使用默认值")  
        fps = 30  
        total_frames = 300  
        duration = total_frames / fps  
  
    print(f"视频信息:")
```

```

print(f" 总帧数: {total_frames}")
print(f" 帧率: {fps}")
print(f" 时长: {duration:.2f}s")

# 计算每个 episode 的帧范围
episodes = []
for idx, row in episode_df.iterrows():
    start_frame = row['start_index']
    end_frame = row['end_index']
    length = row['length']
    episode_idx = row['episode_index']

    # 计算时间 (秒)
    start_time = start_frame / fps
    end_time = (end_frame + 1) / fps # 包含结束帧

    episodes.append({
        'episode_index': episode_idx,
        'start_frame': start_frame,
        'end_frame': end_frame,
        'start_time': start_time,
        'end_time': end_time,
        'duration': end_time - start_time,
        'frame_count': end_frame - start_frame + 1
    })

print(f"\nEpisode 信息:")
for ep in episodes:
    print(f"  Episode {ep['episode_index']}: 帧 {ep['start_frame']}-{ep['end_frame']},
"
        f"时长 {ep['duration']:.2f}s")

# 创建新的视频目录结构
new_video_dir = dataset_path / "videos_fixed" / "observation.images.side"
new_video_dir.mkdir(parents=True, exist_ok=True)

```

```

# 分割视频

for ep in episodes:
    output_file = new_video_dir / f"episode_{ep['episode_index']:03d}.mp4"

    print(f"\n 分割 Episode {ep['episode_index']}...")
    print(f"   输出文件: {output_file}")
    print(f"   时间范围: {ep['start_time']:.2f}s - {ep['end_time']:.2f}s")

    try:
        # 使用 ffmpeg 分割视频
        (
            ffmpeg
            .input(str(input_video), ss=ep['start_time'], t=ep['duration'])
            .output(str(output_file), vcodec='copy', acodec='copy')
            .run(quiet=True, overwrite_output=True)
        )
        print(f"   ✓ 分割成功")
    except Exception as e:
        print(f"   ✗ 分割失败: {e}")

# 更新 info.json 中的视频路径模板
info['video_path'] = 'videos_fixed/{video_key}/episode_{episode_index:03d}.mp4'

# 创建新的 chunk 目录结构
for ep in episodes:
    chunk_dir = dataset_path / "videos_fixed" / "observation.images.side" / f"chunk-
{ep['episode_index']:03d}"
    chunk_dir.mkdir(parents=True, exist_ok=True)

    # 移动视频文件
    src_file = new_video_dir / f"episode_{ep['episode_index']:03d}.mp4"
    dst_file = chunk_dir / f"file-{ep['episode_index']:03d}.mp4"
    shutil.move(src_file, dst_file)

# 清理临时目录
shutil.rmtree(new_video_dir)

```

```

# 更新 info.json 为新的结构
info['video_path'] = 'videos_fixed/{video_key}/chunk-{chunk_index:03d}/file-
{file_index:03d}.mp4'

# 保存更新的 info.json
with open(meta_path, 'w', encoding='utf-8') as f:
    json.dump(info, f, indent=2, ensure_ascii=False)

print(f"\n 视频分割完成!")
print(f"新的视频目录: {dataset_path / 'videos_fixed'}")

return True

def fix_video_encoding_in_record():
    """修改录制时的视频编码逻辑"""
    # 找到 lerobot 的安装路径
    import lerobot
    lerobot_path = Path(lerobot.__file__).parent

    print(f"LeRobot 路径: {lerobot_path}")

    # 查找相关的录制和编码文件
    record_files = list(lerobot_path.glob("**/*record*.py"))
    dataset_files = list(lerobot_path.glob("**/*dataset*.py"))

    print(f"找到录制相关文件: {[f.name for f in record_files]}")
    print(f"找到数据集相关文件: {[f.name for f in dataset_files]}")

    # 主要需要修改的文件可能是 lerobot_dataset.py
    dataset_file = lerobot_path / "datasets" / "lerobot_dataset.py"

    if dataset_file.exists():
        print(f"\n 主要数据集文件: {dataset_file}")

    # 备份原文件

```



```

        backup_file = dataset_file.with_suffix('.py.backup')

        if not backup_file.exists():
            shutil.copy2(dataset_file, backup_file)
            print(f"已备份原文件到: {backup_file}")

    # 读取文件内容
    with open(dataset_file, 'r', encoding='utf-8') as f:
        content = f.read()

    # 查找视频编码相关的方法
    if '_save_episode_video' in content:
        print("找到 _save_episode_video 方法")

    # 我们可以创建一个补丁文件
    patch_content = '''
# 修改视频文件命名逻辑 - 每个 episode 独立文件
def _save_episode_video_patched(self, video_key, episode_index, temp_path=None):
    """
    Encode a video from a sequence of images for a given episode.
    Each episode gets its own video file.
    """

    # 使用 episode_index 作为 file_index
    file_index = episode_index
    chunk_index = 0 # 可以根据需要计算 chunk_index

    video_filename = f"file-{file_index:03d}.mp4"
    video_path = self.root / "videos" / video_key / f"chunk-{chunk_index:03d}" /
video_filename

    # 确保目录存在
    video_path.parent.mkdir(parents=True, exist_ok=True)

    # 如果提供了临时路径（并行编码），使用它
    if temp_path is not None:
        import shutil
        shutil.move(temp_path, video_path)

```

```

else:

    # 否则直接编码

    from .video_utils import encode_images_to_video

    encode_images_to_video(

        episode_index=episode_index,

        video_key=video_key,

        output_path=video_path,

        fps=self.fps,

        root=self.root

    )

# 返回视频元数据

return {

    f"{video_key}.path": str(video_path.relative_to(self.root)),

    f"{video_key}.chunk_index": chunk_index,

    f"{video_key}.file_index": file_index,

    f"{video_key}.timestamps": [

        self.episode_buffer["timestamp"][0],

        self.episode_buffer["timestamp"][-1],

    ],

    f"{video_key}.frame_index": [

        self.episode_buffer["frame_index"][0],

        self.episode_buffer["frame_index"][-1],

    ],

}

'''

    print("已创建补丁内容")

    # 保存补丁文件

    patch_file = dataset_file.parent / "video_fix_patch.py"

    with open(patch_file, 'w', encoding='utf-8') as f:

        f.write(patch_content)

    print(f"补丁文件保存到: {patch_file}")

return dataset_file

```

```
def create_fixed_dataset_structure(dataset_path):
    """创建修正后的数据集结构"""
    dataset_path = Path(dataset_path)

    # 读取元数据
    meta_path = dataset_path / "meta" / "info.json"
    with open(meta_path, 'r', encoding='utf-8') as f:
        info = json.load(f)

    # 读取 episode_data
    import pandas as pd
    episode_data_path = dataset_path / "meta" / "episode_data.parquet"
    episode_df = pd.read_parquet(episode_data_path)

    # 创建新的视频目录结构
    new_video_base = dataset_path / "videos_by_episode"
    new_video_base.mkdir(exist_ok=True)

    video_key = "observation.images.side"
    video_dir = new_video_base / video_key
    video_dir.mkdir(exist_ok=True)

    # 假设原始视频文件路径
    original_video = dataset_path / "videos" / video_key / "chunk-000" / "file-000.mp4"

    if not original_video.exists():
        print(f"原始视频文件不存在: {original_video}")
        return False

    # 为每个 episode 创建视频文件（这里只是创建空文件，实际需要分割视频）
    for idx, row in episode_df.iterrows():
        episode_idx = row['episode_index']

        # 创建 chunk 目录（每个 episode 一个 chunk）
        chunk_dir = video_dir / f"chunk-{episode_idx:03d}"
```

```

    chunk_dir.mkdir(exist_ok=True)

    # 创建视频文件占位符
    video_file = chunk_dir / f"file-{episode_idx:03d}.mp4"
    video_file.touch()

    print(f"为 Episode {episode_idx} 创建视频文件: {video_file}")

    # 更新 info.json 中的视频路径
    info['video_path'] = 'videos_by_episode/{video_key}/chunk-{chunk_index:03d}/file-{file_index:03d}.mp4'

    # 保存更新后的 info.json
    with open(meta_path, 'w', encoding='utf-8') as f:
        json.dump(info, f, indent=2, ensure_ascii=False)

    print(f"\n 已创建新的视频目录结构: {new_video_base}")
    print(f"每个 episode 现在有独立的视频文件路径")

    return True

def main():
    """主函数"""
    dataset_path = r"E:/WT/clean_test_final"

    print("=" * 60)
    print("修复视频分段问题")
    print("=" * 60)

    print("1. 尝试分割现有视频文件...")
    success = split_video_by_episodes(dataset_path)

    if not success:
        print("\n2. 创建修正后的数据集结构...")
        create_fixed_dataset_structure(dataset_path)

```

```

print("\n3. 检查修改录制代码的可能性...")
fix_video_encoding_in_record()

print("\n" + "=" * 60)
print("修复完成!")
print("=" * 60)
print("\n 注意事项:")
print("1. 如果视频已成功分割, 新视频文件在 videos_fixed/ 目录下")
print("2. 下次录制时, 建议修改录制代码以生成独立的视频文件")
print("3. 可以尝试修改 LeRobot 源代码中的视频编码逻辑")

if __name__ == "__main__":
    # 确保安装了 ffmpeg-python
    try:
        import ffmpeg
    except ImportError:
        print("请先安装 ffmpeg-python: pip install ffmpeg-python")
        exit(1)

    main()

```

运行录制配置修复脚本

python fix_recording_config.py

```

# fix_recording_config.py
import yaml
from pathlib import Path

def create_fixed_config():
    """创建修正后的录制配置"""
    config = {
        'teleop': {
            'type': 'so101_leader',
            'port': 'COM22',

```

```

        'id': 'my_awesome_leader_arm'
    },
    'robot': {
        'type': 'sol101_follower',
        'port': 'COM7',
        'id': 'my_awesome_follower_arm',
        'disable_torque_on_disconnect': True,
        'cameras': {
            'side': {
                'type': 'opencv',
                'index_or_path': 0,
                'width': 640,
                'height': 480,
                'fps': 30,
                'fourcc': 'YUY2'
            }
        }
    },
    'dataset': {
        'root': 'E:/WT/clean_test_separate_videos', # 新的数据集路径
        'repo_id': 'luoxingtong/record_test_separate',
        'num_episodes': 2,
        'single_task': '抓绿油板',
        'push_to_hub': False,
        'episode_time_s': 5,
        'reset_time_s': 2,
        'video': True,
        'video_encoding_batch_size': 1, # 每个 episode 单独编码
        # 添加视频编码配置
        'video_encoding': {
            'separate_files_per_episode': True, # 每个 episode 单独文件
            'chunk_size': 1, # 每个 chunk 包含 1 个 episode
            'naming_pattern': 'episode_{episode_index:03d}.mp4'
        }
    },
    'display_data': True,

```

```

        'resume': False
    }

    # 保存配置文件
    config_path = Path("ceshi_test_fixed.yaml")
    with open(config_path, 'w', encoding='utf-8') as f:
        yaml.dump(config, f, default_flow_style=False, allow_unicode=True)

    print(f"已创建修正后的配置文件: {config_path}")
    return config_path

def modify_existing_config():
    """修改现有配置文件"""
    config_path = Path("ceshi_test.yaml")

    if not config_path.exists():
        print(f"配置文件不存在: {config_path}")
        return create_fixed_config()

    # 读取现有配置
    with open(config_path, 'r', encoding='utf-8') as f:
        config = yaml.safe_load(f)

    # 修改视频编码相关配置
    if 'dataset' not in config:
        config['dataset'] = {}

    # 确保每个 episode 有独立的视频文件
    config['dataset']['video_encoding_batch_size'] = 1

    # 添加视频编码配置
    config['dataset']['video_encoding'] = {
        'separate_files_per_episode': True,
        'chunk_size': 1,
        'naming_pattern': 'episode_{episode_index:03d}.mp4'
    }

```

```

# 保存修改后的配置
new_config_path = Path("ceshi_test_modified.yaml")
with open(new_config_path, 'w', encoding='utf-8') as f:
    yaml.dump(config, f, default_flow_style=False, allow_unicode=True)

print(f"已修改配置文件: {new_config_path}")
return new_config_path

def create_monkey_patch():
    """创建猴子补丁来修改 LeRobot 的视频编码行为"""
    patch_code = '''
# monkey_patch_video_encoding.py
import sys
import os
from pathlib import Path

# 添加当前目录到 Python 路径
sys.path.insert(0, str(Path(__file__).parent))

# 尝试导入 LeRobot 并修改视频编码
try:
    import lerobot.datasets.lerobot_dataset as dataset_module

    # 保存原始方法
    original_save_episode_video = dataset_module.LeRobotDataset._save_episode_video

    def patched_save_episode_video(self, video_key, episode_index, temp_path=None):
        """
        修改后的视频保存方法，每个 episode 生成独立的视频文件
        """

        # 使用 episode_index 作为 file_index
        file_index = episode_index
        chunk_index = episode_index // 1000 # 每 1000 个 episode 一个 chunk

        video_filename = f"file-{file_index:03d}.mp4"

```



```

        video_path = self.root / "videos" / video_key / f"chunk-{chunk_index:03d}" /
video_filename

        # 确保目录存在
        video_path.parent.mkdir(parents=True, exist_ok=True)

        # 如果提供了临时路径（并行编码），使用它
        if temp_path is not None:
            import shutil
            shutil.move(temp_path, video_path)
        else:
            # 否则调用原始方法，但传入修改后的路径
            # 这里需要找到原始编码逻辑并修改
            pass

        # 返回视频元数据
        return {
            f"{video_key}.path": str(video_path.relative_to(self.root)),
            f"{video_key}.chunk_index": chunk_index,
            f"{video_key}.file_index": file_index,
            f"{video_key}.timestamps": [
                self.episode_buffer["timestamp"][0],
                self.episode_buffer["timestamp"][-1],
            ],
            f"{video_key}.frame_index": [
                self.episode_buffer["frame_index"][0],
                self.episode_buffer["frame_index"][-1],
            ],
        }

    # 应用补丁
    dataset_module.LeRobotDataset._save_episode_video = patched_save_episode_video

    print("✓ 已应用视频编码补丁：每个 episode 将生成独立的视频文件")

except Exception as e:

```

```

print(f"✗ 应用补丁失败: {e}")

import traceback

traceback.print_exc()
'''

patch_path = Path("monkey_patch_video_encoding.py")
with open(patch_path, 'w', encoding='utf-8') as f:
    f.write(patch_code)

print(f"已创建猴子补丁文件: {patch_path}")
print("在录制前运行: python monkey_patch_video_encoding.py")

def main():
    print("=" * 60)
    print("修复录制配置")
    print("=" * 60)

    print("\n1. 创建修正后的录制配置...")
    config_path = modify_existing_config()

    print("\n2. 创建猴子补丁...")
    create_monkey_patch()

    print("\n" + "=" * 60)
    print("配置修复完成!")
    print("=" * 60)
    print("\n使用方法:")
    print(f"1. 使用新的配置文件录制: lerobot-record --config_path={config_path}")
    print("2. 或者在录制前运行猴子补丁: python monkey_patch_video_encoding.py")
    print("\n注意:")
    print("- 每个 episode 现在应该有独立的视频文件")
    print("- 视频文件名将包含 episode_index")
    print("- chunk 目录将按 episode_index // 1000 分组")

if __name__ == "__main__":

```

```
main()
```

#简单分割脚本(以上方法不行，可以尝试如下)

```
python simple_video_split.py
```

```
# simple_video_split.py
import subprocess
import json
from pathlib import Path

def split_video_simple(dataset_path):
    """简单分割视频文件"""
    dataset_path = Path(dataset_path)

    # 读取 episode 信息
    import pandas as pd
    episode_df = pd.read_parquet(dataset_path / "meta" / "episode_data.parquet")

    # 原始视频文件
    input_video = dataset_path / "videos" / "observation.images.side" / "chunk-000" /
    "file-000.mp4"

    # 每个 episode 的帧数（假设都是 150 帧）
    frames_per_episode = 150
    fps = 30

    for idx, row in episode_df.iterrows():
        episode_idx = row['episode_index']

        # 计算时间点
        start_time = episode_idx * frames_per_episode / fps
        duration = frames_per_episode / fps

        # 输出文件
```

```

        output_dir = dataset_path / "videos_split" / "observation.images.side" / f"chunk-
{episode_idx:03d}"

        output_dir.mkdir(parents=True, exist_ok=True)

        output_file = output_dir / f"file-{episode_idx:03d}.mp4"

    # 使用 ffmpeg 分割
    cmd = [
        'ffmpeg',
        '-i', str(input_video),
        '-ss', str(start_time),
        '-t', str(duration),
        '-c', 'copy',
        str(output_file),
        '-y' # 覆盖已存在的文件
    ]

    print(f"分割 Episode {episode_idx}: {start_time:.2f}s -
{start_time+duration:.2f}s")

    try:
        subprocess.run(cmd, check=True, capture_output=True)
        print(f"    ✓ 成功: {output_file}")
    except subprocess.CalledProcessError as e:
        print(f"    ✗ 失败: {e}")
        print(f"    错误输出: {e.stderr.decode()}")

def main():
    dataset_path = r"E:/WT/clean_test_final"
    split_video_simple(dataset_path)

if __name__ == "__main__":
    main()

```